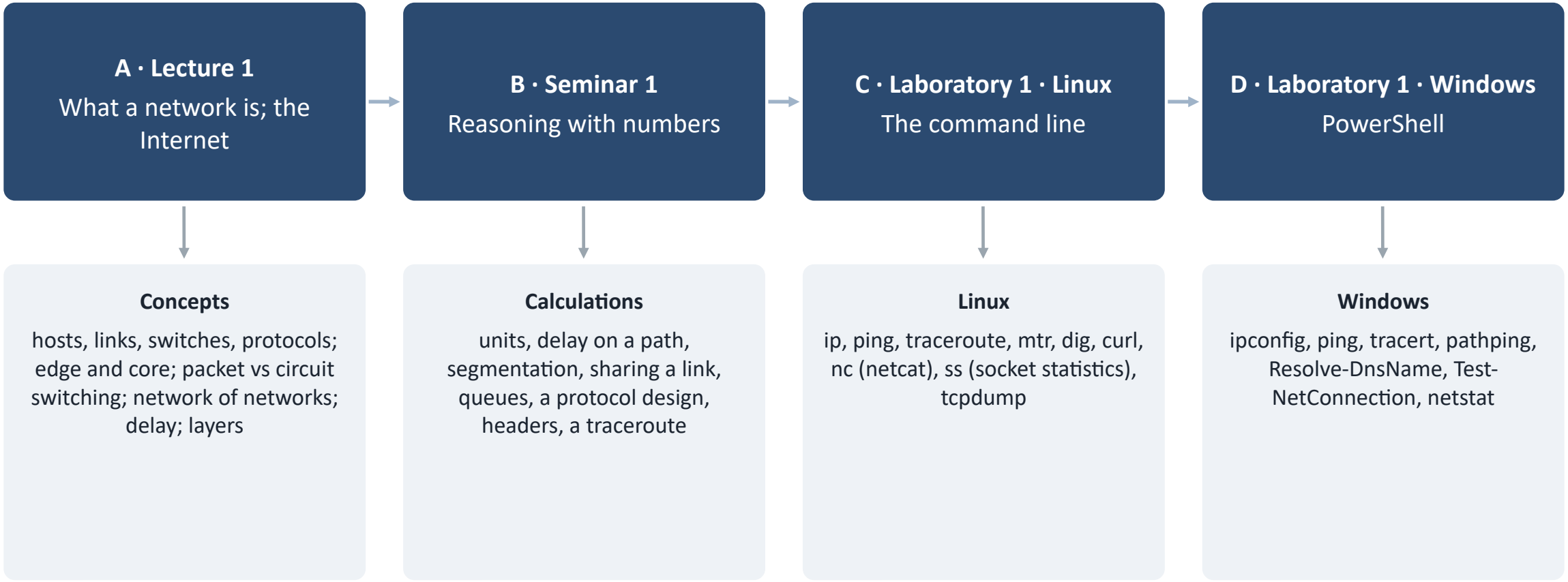
COMPUTER NETWORKS · SEMESTER 1 · WEEK 1

CN Introduction in schemes

What a network is and how the Internet is put together — lecture, seminar and laboratories

Assoc. Prof. PhD. Nicolaie Popescu-Bodorin, 2026
+40722882471 · bodorin(on)live.com · https://lmrec.org/bodorin/

How this week is organised



Key idea The lecture names the parts of the Internet; the laboratories find each of them on your own machine.

mtr (my traceroute) · dig (domain information groper)

Part A · Lecture 1: introduction

A · Lecture 1
What a network is; the Internet

→

B · Seminar 1
Reasoning with numbers

→

C · Laboratory 1 · Linux
The command line

→

D · Laboratory 1 · Windows
PowerShell

In this part

1 · Why study networks; the course

2 · What is a network? The Internet

3 · What is a protocol?

4 · The network edge

5 · The core: packet and circuit switching

6 · A network of networks

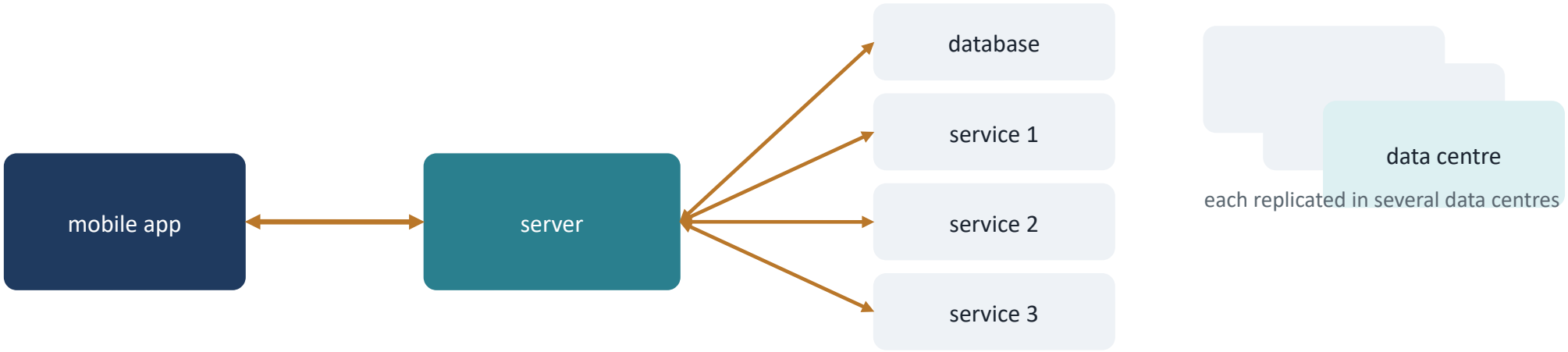
7 · A first look at delay

8 · Layering in one picture

9 · A short history

Summary · review · exercises

1 · Why study networks: the network is between every part



1. Every **amber arrow** is a network path: slow, lossy or broken in ways the programs cannot see.

2. An engineer who understands it finds the cause; one who treats it as a black box can only restart and hope.

The second reason: the largest engineered system

billions of devices · owned by nobody · no central control

grew by many orders of magnitude, carrying applications nobody planned

ideas: **layering · packet switching · best-effort delivery · intelligence at the edges**

Key idea Understanding the network between the parts turns “restart and hope” into finding the cause.

1 · How the course is organised

Semester 1 — down the stack from the top

applications → transport → network layer and routing

Semester 2

- link and physical layers
- wireless and mobile
- security
- software-defined and data-centre networks

Laboratory (compulsory)
write network programs and take networks apart
Linux · C · Python · Wireshark

Books
Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed. (main)
Peterson & Davie, *Computer Networks: A Systems Approach* (free online)

Assessment, each semester



at least **5** needed in both the examination and the laboratory

Key idea Semester 1 goes top-down through the stack; the laboratory is where the ideas become programs.

2 · What is a computer network? What is the Internet?

computer network = computing devices connected so that they can exchange data



two laptops
joined by a cable

this building
its Wi-Fi

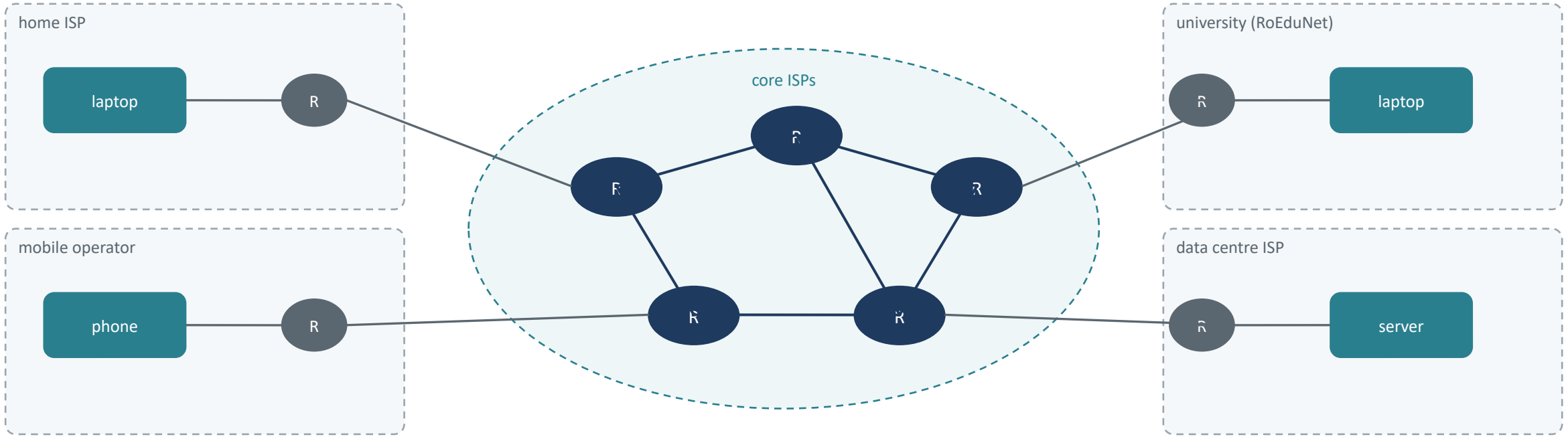
a mobile operator
a national network

the Internet
the whole world

The Internet: a worldwide **network of networks** — independently owned networks agree on one protocol family, **TCP/IP** (Transmission Control Protocol / Internet Protocol),
so any device on any of them can reach any device on any other.

Key idea From two laptops to the whole planet: the same definition, at every scale.

2.1 · The nuts-and-bolts view: what the Internet is made of



1. **hosts** (end systems) run applications
2. **links**: copper, fibre, radio — rate in bit/s
3. **packet switches**: routers (R), link-layer switches
4. **ISPs** (Internet service providers): networks of switches and links
5. **protocols** (IP, TCP) and **standards**: IETF RFCs, IEEE 802

Key idea Hosts, links, packet switches, ISPs, protocols, standards; the links and switches a packet crosses form its route.

RoEduNet (Romanian Education Network) · IETF (Internet Engineering Task Force) · IEEE (Institute of Electrical and Electronics Engineers) · TCP (Transmission Control Protocol) · RFC (Request for Comments) · IP (Internet Protocol)

2.1 · From data to packets, from packets to a route



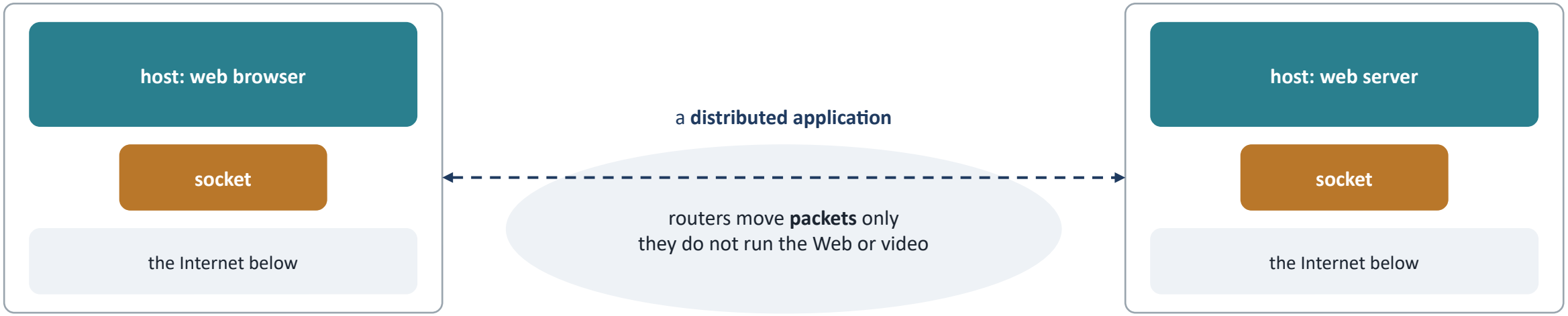
packet = header (destination address, …) + a chunk of data



1. Each switch: **receive** the packet on one link, **forward** it on another.
2. The sequence of links and switches (amber) is the packet's **route** or **path**.

Key idea Packets are self-addressed chunks; switches pass them on link by link along a route.

2.2 · The service view: what the Internet does for applications



The socket interface is like the postal rules

envelope
put the data in a message

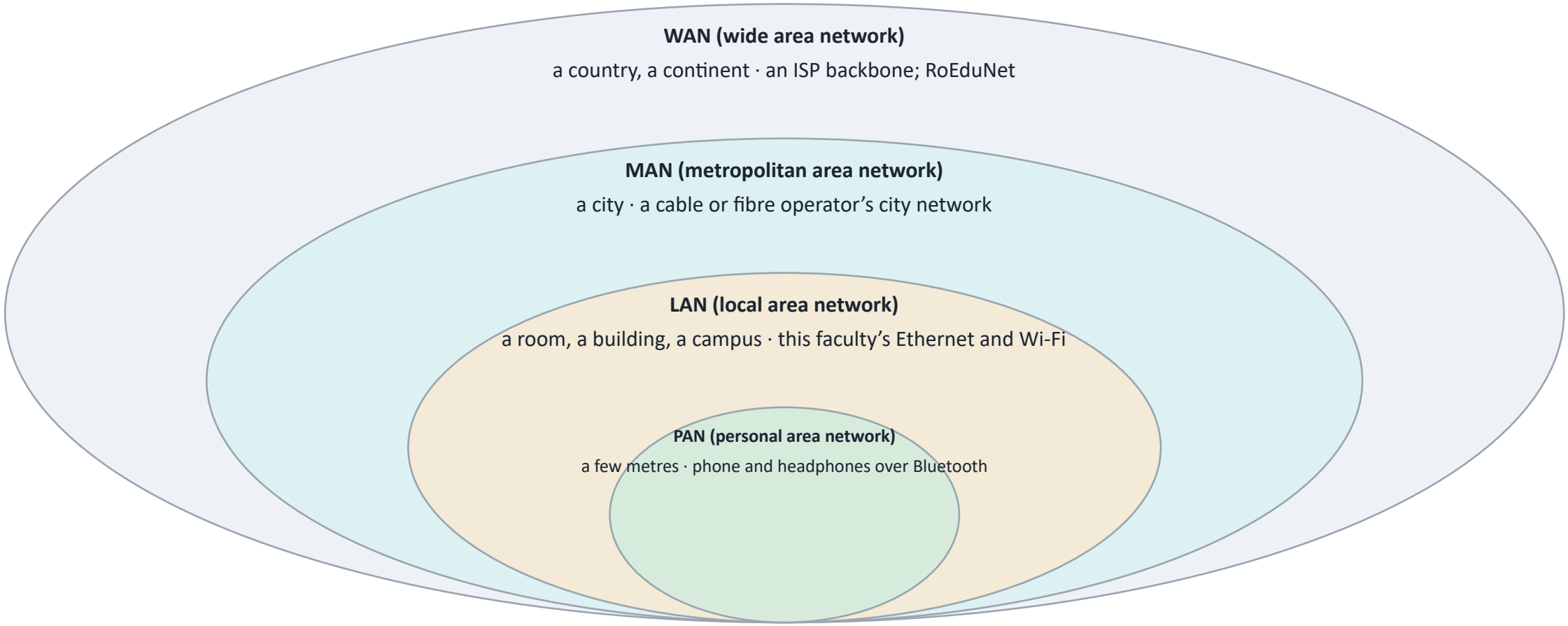
address format
write the destination in a fixed form

post box
hand it to the service

ordinary · registered · express
the Internet also offers more than one delivery service

Key idea Applications live on hosts and use the socket interface; choosing the right delivery service is a design decision (sockets: laboratory 3).

2.3 · Classifying networks by geographic scale

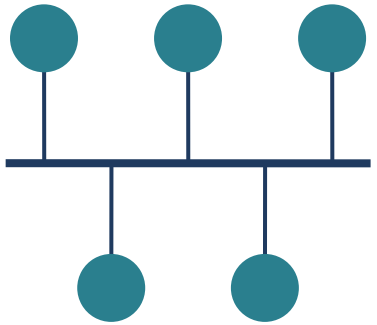


Key idea Scale is the first vocabulary of networking: PAN inside LAN inside MAN inside WAN.

RoEduNet (Romanian Education Network) · ISP (Internet service provider)

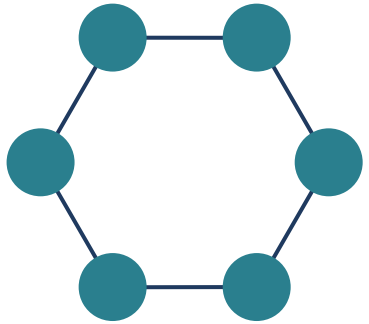
2.3 · Classifying networks by topology

bus



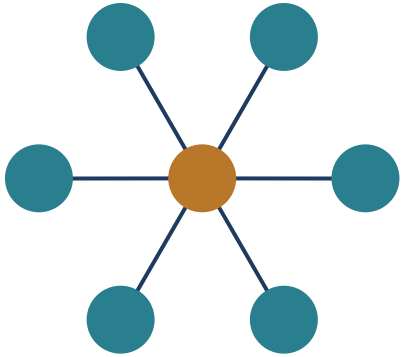
all nodes on one shared cable (early Ethernet)

ring



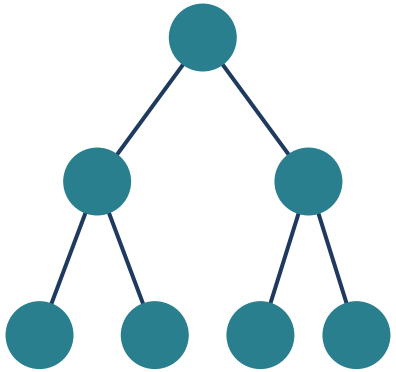
each node linked to two neighbours

star



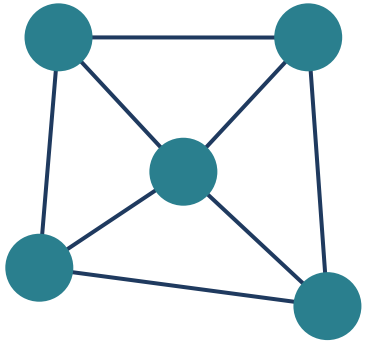
all to a centre: switched Ethernet, Wi-Fi · **lose the centre, lose everyone**

tree



stars of stars, in levels

mesh

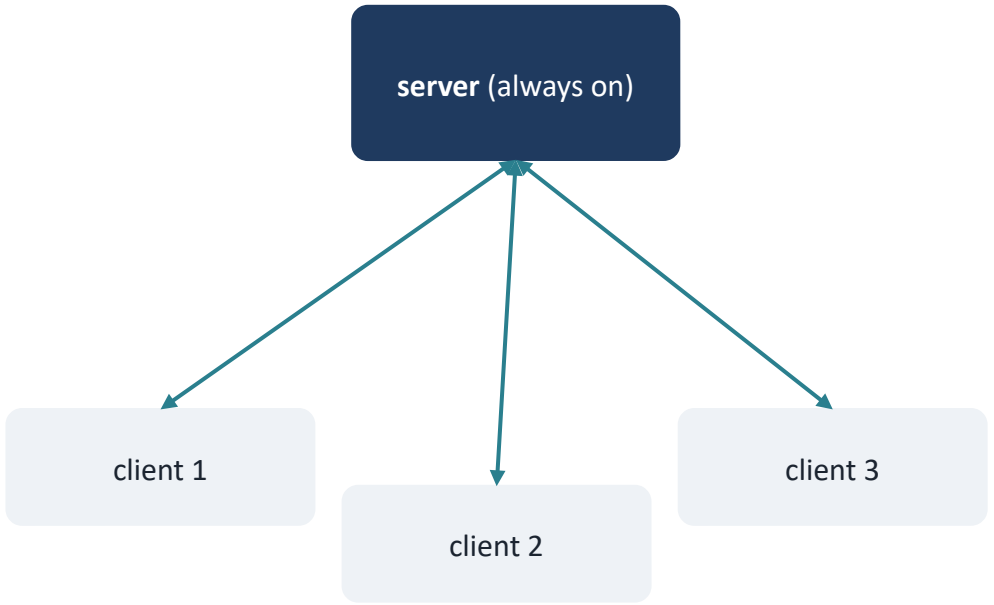


several alternative paths (Internet core) · **traffic goes around a broken link**

Key idea Topology is about failure: a star has a single point of failure, a mesh has alternatives.

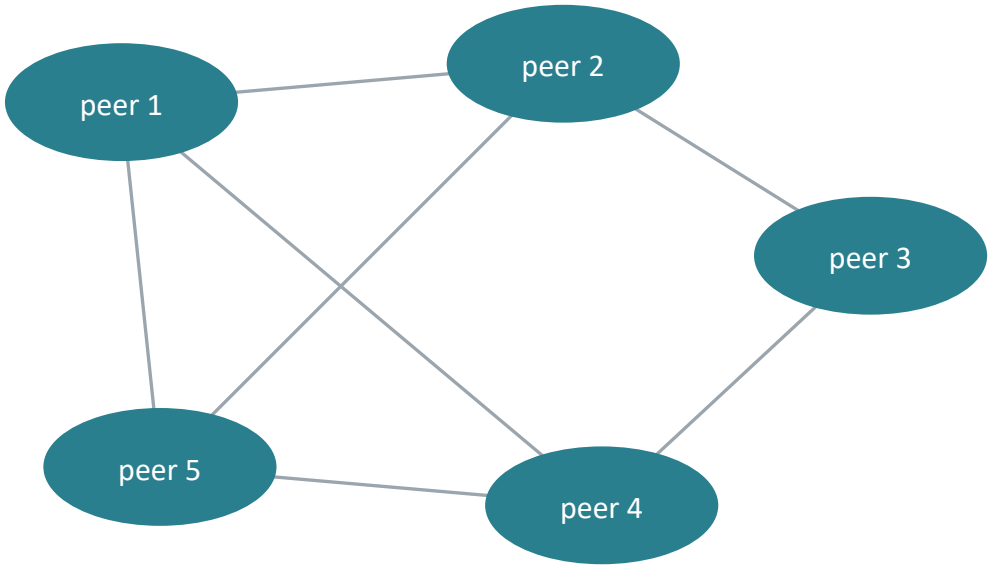
2.3 · Classifying networks by the roles of the hosts

client–server



clients start every exchange · e.g. the Web

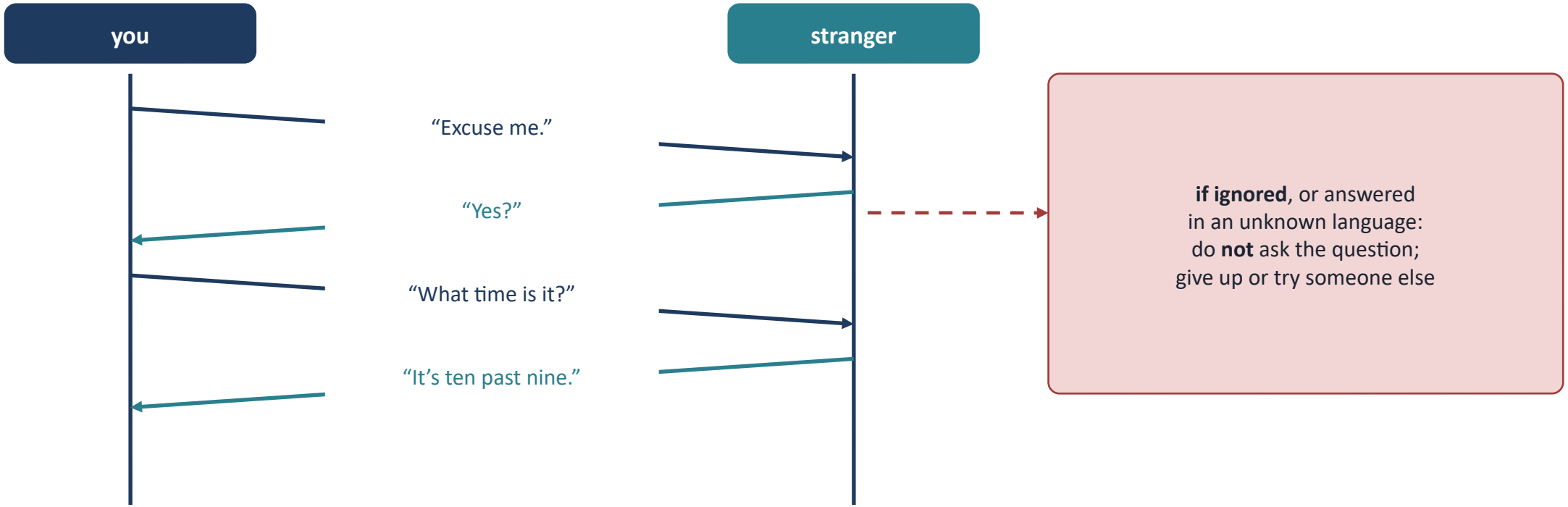
peer-to-peer



each peer is client and server · e.g. BitTorrent

Key idea Who serves whom: a fixed server for many clients, or every host serving every other.

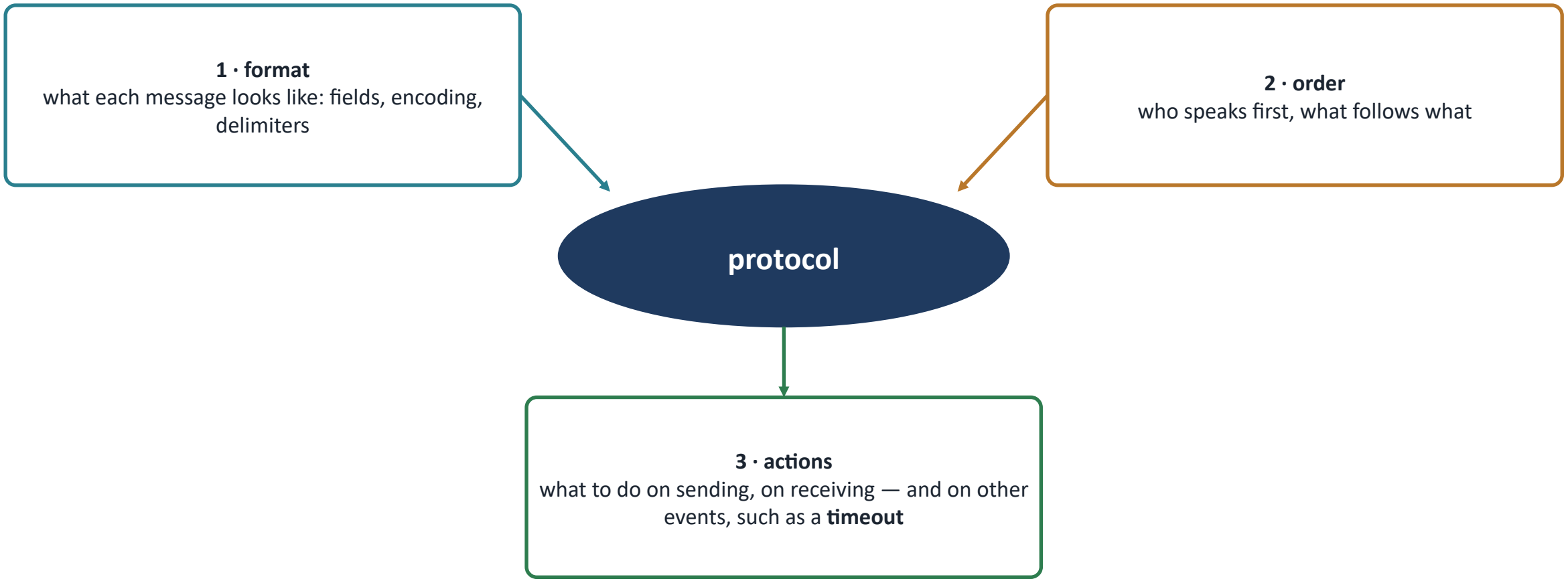
3 · A human protocol: asking a stranger for the time



Machines do the same, with one difference: **nothing can be left to common sense** — everything must be specified.

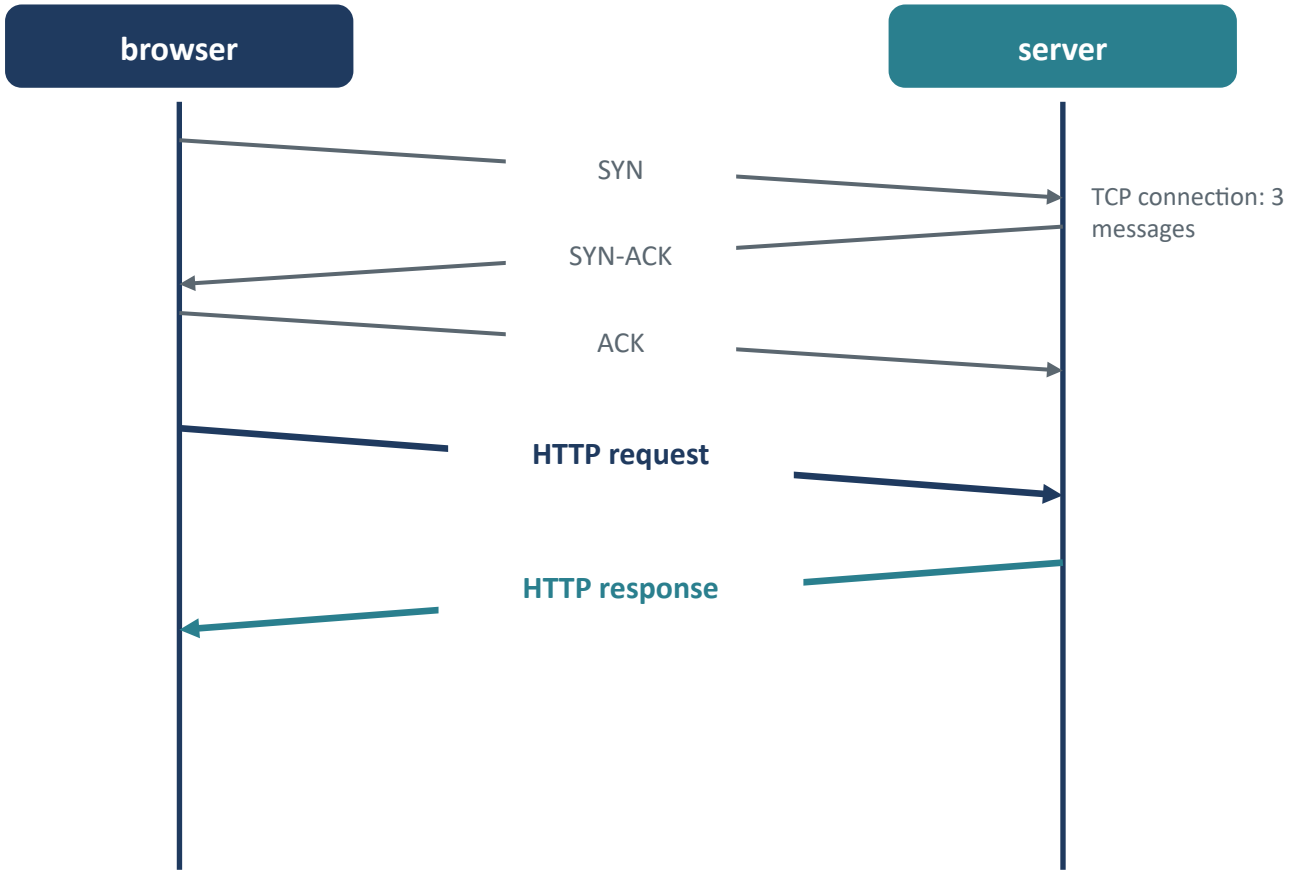
Key idea Messages, their order, and the action each one (or its absence) triggers: that is a protocol.

3 · The definition: format, order, actions



Key idea Format, order, actions: a checklist for reading — or designing — any protocol.

3 · A real protocol: fetching a web page




```
GET /index.html HTTP/1.1
Host: www.example.org
```

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256
<!doctype html> …
```

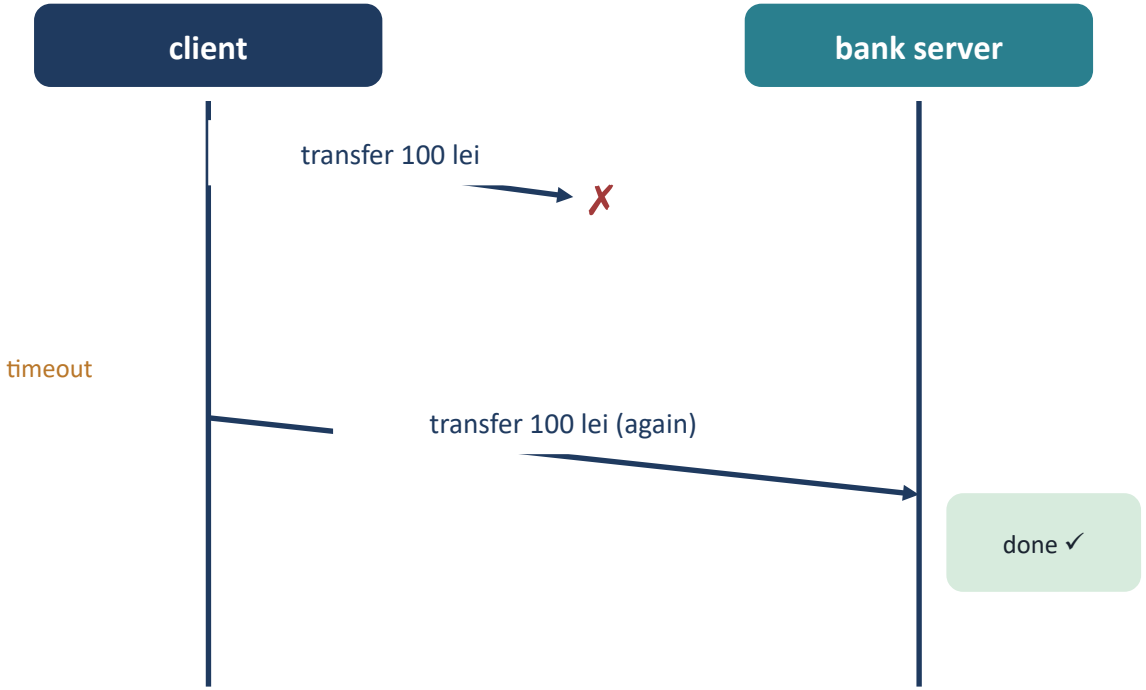
1 Fixed by the standard: the word GET, the spaces, the version string, the line endings, the meaning of **200**.

Key idea Two programs by strangers in different countries interoperate because both followed the same document.

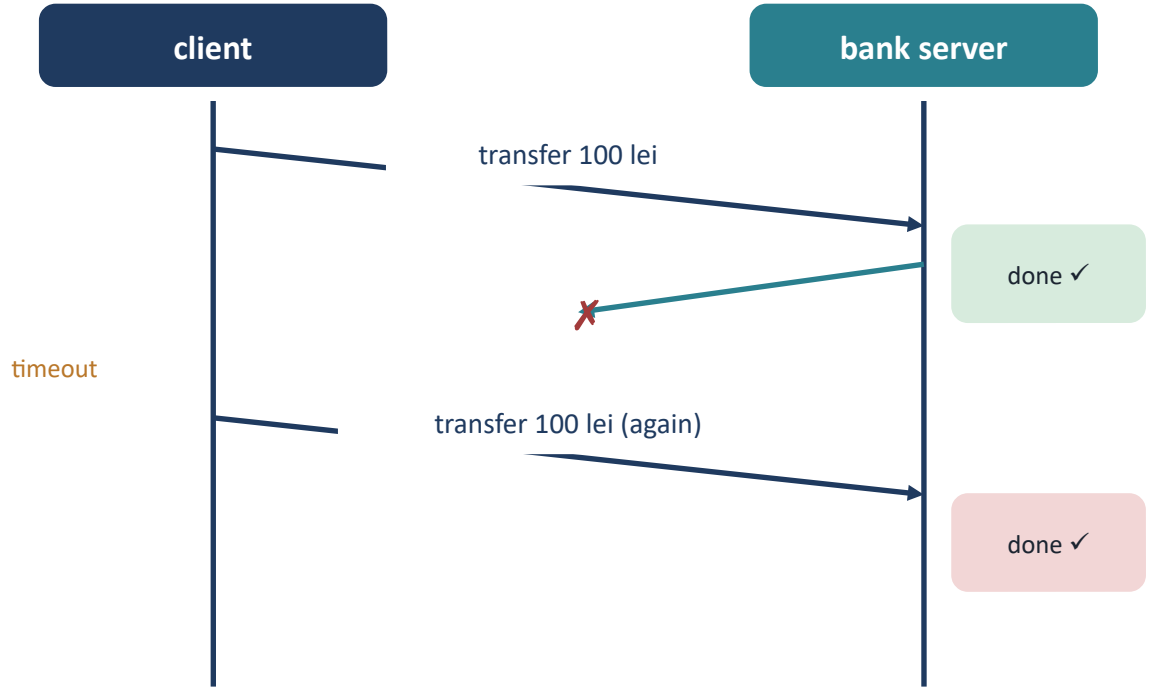
HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · SYN (synchronize) · ACK (acknowledgement)

3 · Question for the room: what if no reply arrives?

(a) the request was lost



(b) the reply was lost



request lost → resending is **correct**: one transfer

reply lost → resending makes **two transfers ✗**

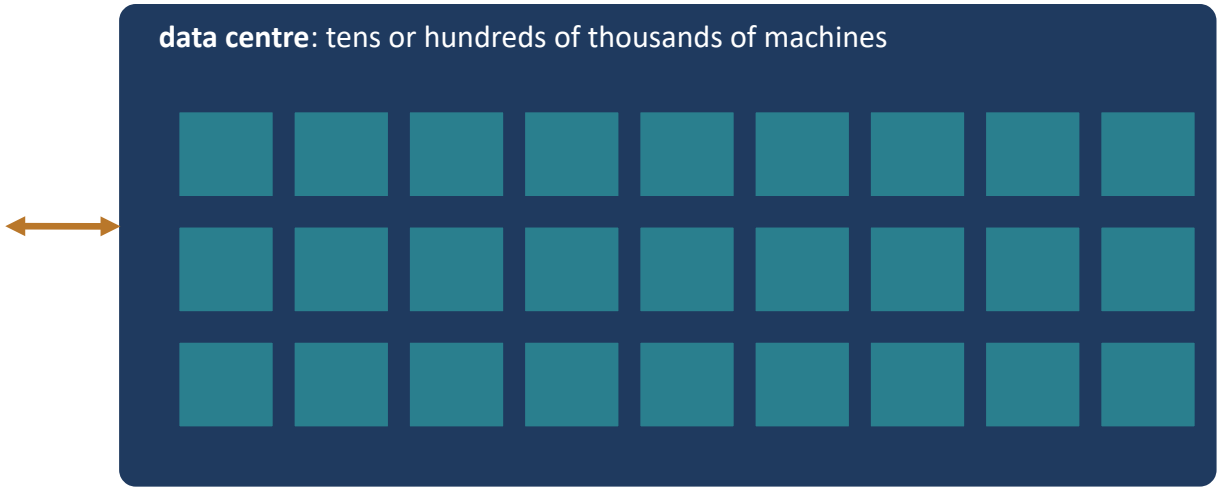
Key idea After how long should the client resend, and how does the server recognise a repetition? Keep this question in mind.

4.1 · The edge: hosts, clients and servers

laptop | phone | television

car | sensor | door lock

clients — used directly by people



servers — store and distribute web pages, video, e-mail

1 “Host” because they **host application programs** — clients and servers together form the **network edge**.

Key idea The edge is where applications run: people’s devices on one side, data centres on the other.

4.2 · Access networks: from a host to the edge router

DSL (digital subscriber line)

existing telephone pair; voice and data on different frequencies; rate falls with distance

tens of Mbit/s down, less up

Cable (hybrid fibre–coax)

fibre to a neighbourhood node, coax to homes — a **shared medium**

hundreds of Mbit/s – ≈ 1 Gbit/s down

FTTH (fibre to the home)

fibre all the way; passive optical network: one fibre split among homes; common in Romanian cities

1 Gbit/s and above

Ethernet

twisted-pair copper from each host to a switch; offices, labs, data centres

1–10 Gbit/s to the desk; up to 400 Gbit/s between switches

Wi-Fi (IEEE 802.11)

radio to an access point within tens of metres; the access point is wired onward

tens of Mbit/s – over 1 Gbit/s

Cellular (4G, 5G)

radio to a base station kilometres away; the operator carries data onward

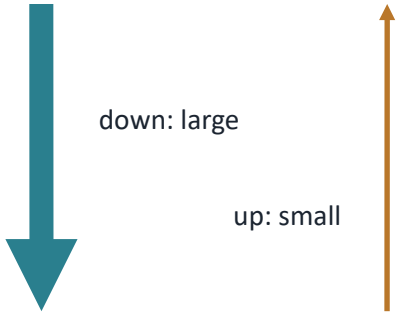
tens – hundreds of Mbit/s

Key idea Wired (DSL, cable, fibre, Ethernet) or wireless (Wi-Fi, cellular): every host reaches its edge router through one of these.

IEEE (Institute of Electrical and Electronics Engineers) · 4G (fourth generation) · 5G (fifth generation)

4.2 · Two facts that do not change, and a typical home

1 · asymmetric

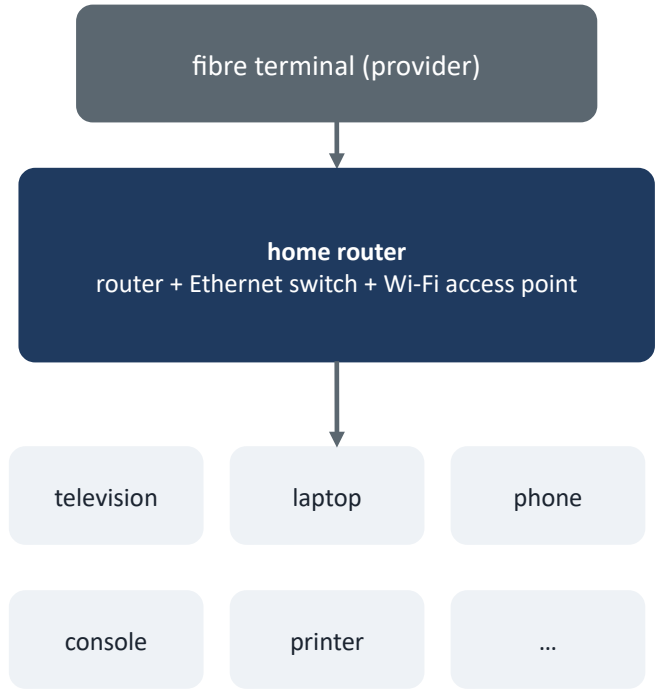


2 · shared



your rate depends on your neighbours

a typical home network

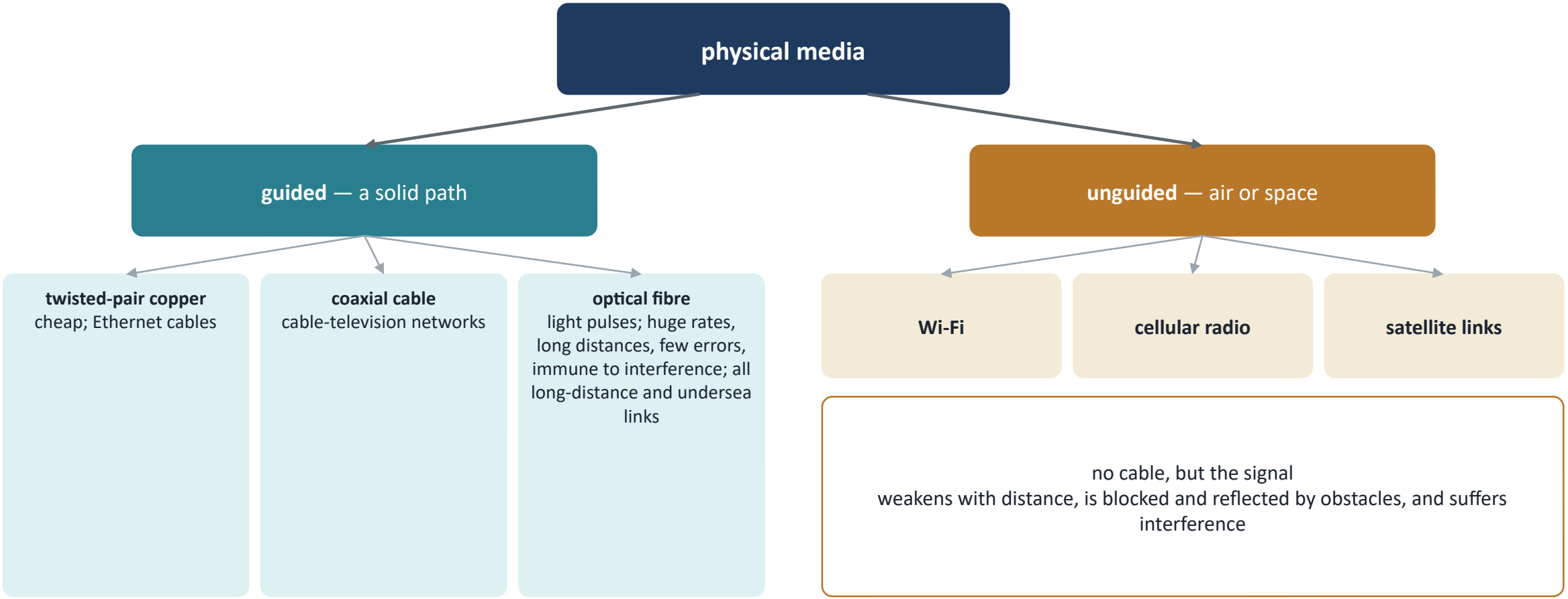


Access links usually give **more capacity towards the user** than away from them.

Many media (cable, Wi-Fi, cellular) are **shared**: what you get depends on what others are doing.

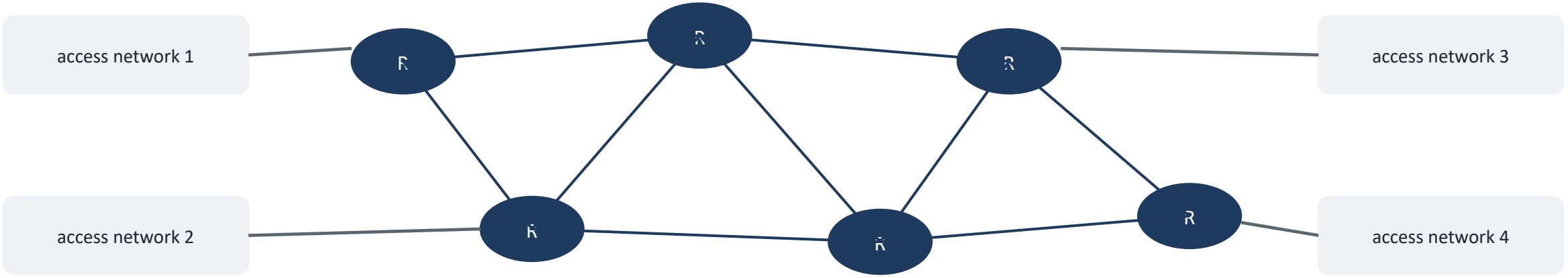
Key idea Remember the shape, not the numbers: asymmetric and often shared.

4.3 · Physical media: guided and unguided



Key idea Signals in fibre and copper travel at about two thirds of light speed: $s \approx 2 \times 10^8$ m/s. (Physical layer: Semester 2.)

5 · The core: a mesh of routers between the access networks

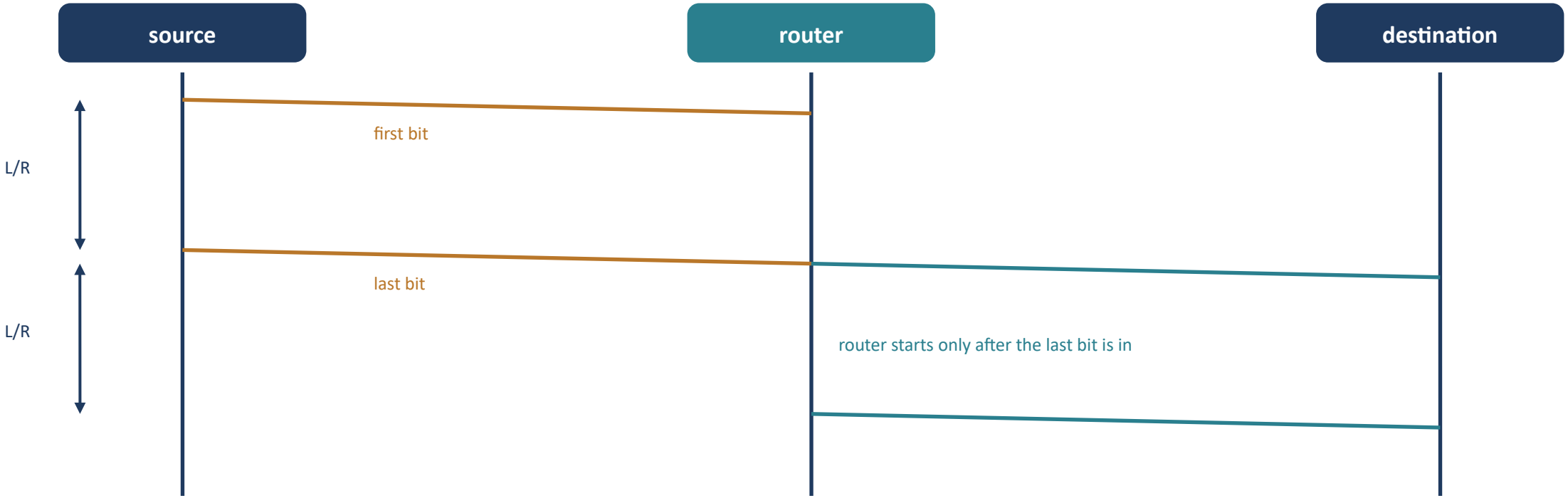


packet switching (§ 5.1)
no reservation; packets share links on demand
the Internet

circuit switching (§ 5.2)
capacity reserved end to end before data flows
the traditional telephone network

Key idea The core's job is to carry data between access networks — by sharing on demand, or by reserving in advance.

5.1 · Store and forward: d = N · L / R

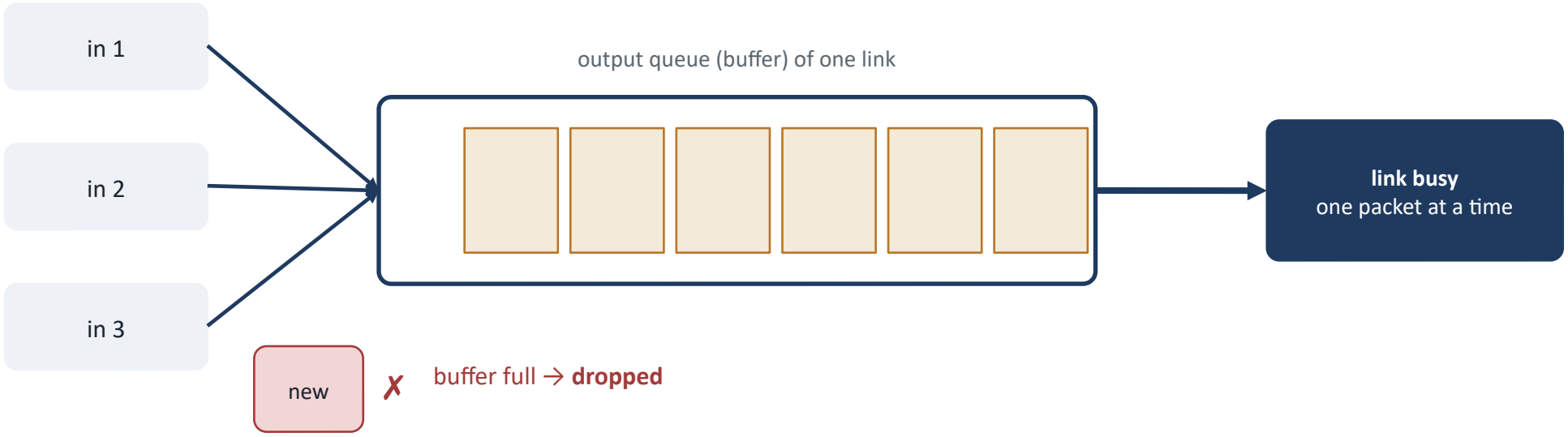


d = N · L / R for N equal links

Worked example 1 · 1500 B = 12 000 bits at 100 Mbit/s: L/R = **0.12 ms**
two links, one router: 2 × 0.12 = **0.24 ms** (propagation ignored)

Key idea Each link costs L/R because the whole packet must be received before it can be forwarded.

5.1 · Queues and loss

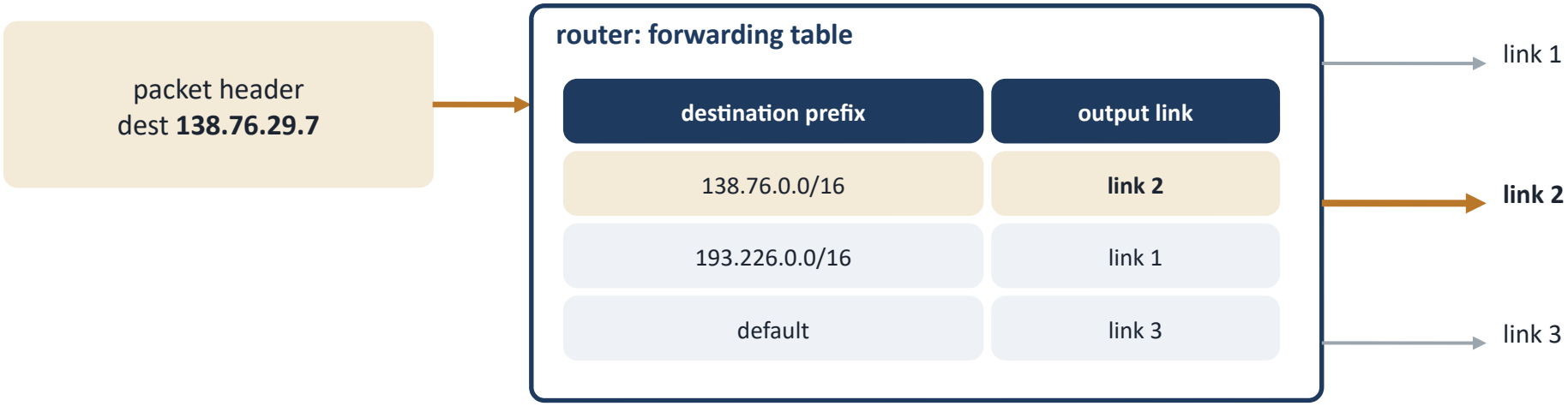


1 **Queueing delay**: waiting for the link; depends on arriving traffic, so it changes from moment to moment.

2 **Packet loss**: buffers are finite. In the Internet loss is the normal result of congestion, not a fault.

Key idea Packet switching shares links without reservation — the price is queueing delay and, when buffers fill, loss.

5.1 · Forwarding and routing



forwarding — local, per packet
look up the destination, send on the link the table gives
≈ following the sign at one junction

routing — global, by routing protocols
routers exchange information and compute good paths
≈ deciding what all the signs to Cluj should say

Key idea Forwarding moves one packet at one router; routing fills every router's table so that forwarding works.

5.2 · Circuit switching: reserve first, then send



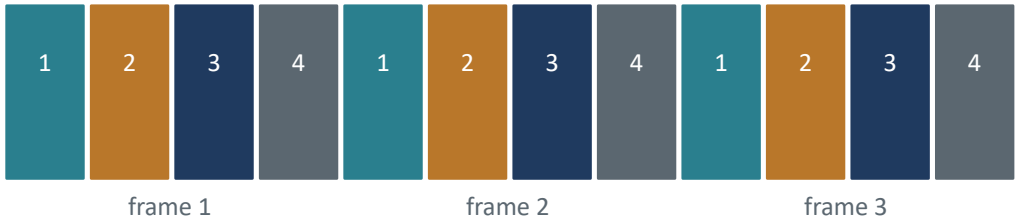
one channel on every link reserved for this call (amber), the others for other calls

FDM (frequency-division multiplexing)



each circuit: its own band, all the time (like radio stations)

TDM (time-division multiplexing)



time → · each circuit: the same slot in every frame

Key idea A circuit is capacity reserved on every link of the path — by frequency band (FDM) or by time slot (TDM).

5.2 · Worked example 2: a reserved slot, used or not



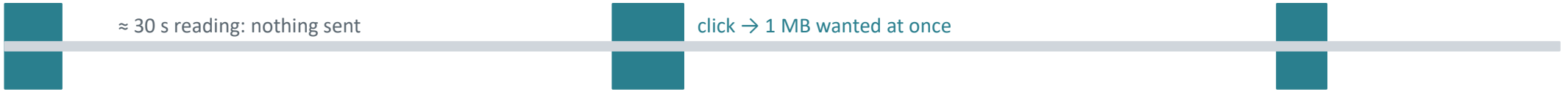
one frame = 32 slots · our circuit = slot 1 · 2 048 000 / 32 = **64 kbit/s** · the other 31 slots stay reserved for others, even when idle



set-up 0.5 s

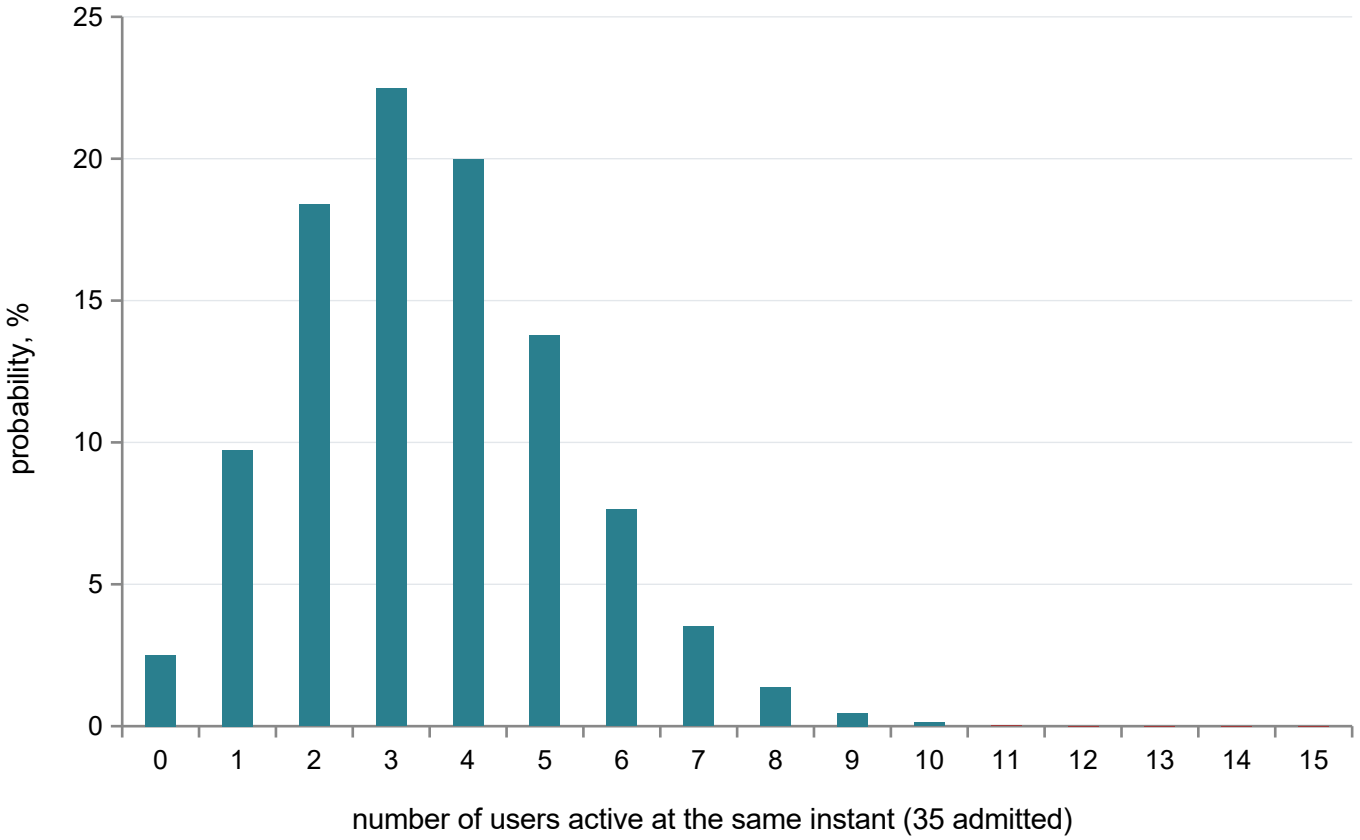
total **10.5 s** — whether the other 31 slots are busy or idle

Bursty traffic — a person reading a web page:



Key idea Circuits fit steady telephone sound; computer traffic is bursty, so reserved capacity sits idle.

5.3 · Statistical multiplexing: worked example 3



circuit switching
100 Mbit/s reserved each → 1000 / 100 = **10 users**

packet switching, 35 users
overload needs ≥ 11 active (> 1 Gbit/s)
$P = \Sigma\, C(35,k)\, 0.1^{k}\, 0.9^{35-k}$, $k \geq 11 \approx$ **0.0004**

1 **3.5 × more users**; overload only ≈ 0.04 % of the time — and then packets just wait briefly in a queue.

Key idea Sharing on demand (statistical multiplexing) serves far more bursty users than reservation can.

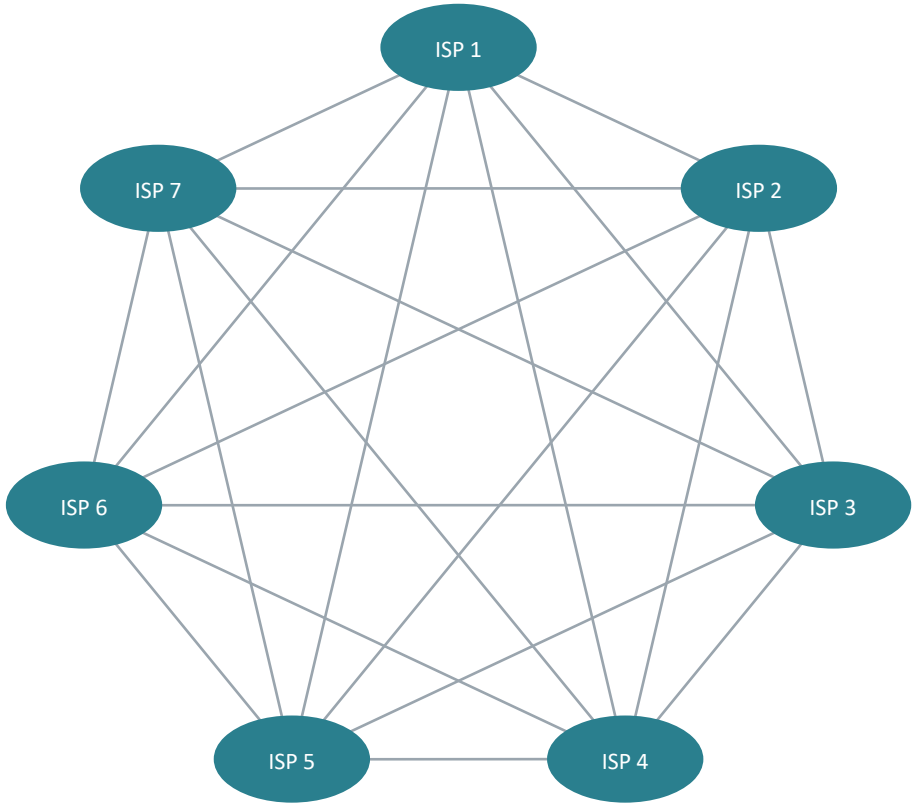
5.3 · Packet switching versus circuit switching

	Packet switching	Circuit switching
set-up before sending	none	required
capacity	shared on demand	reserved per connection
idle capacity	available to others	wasted
delay	variable; queueing possible	constant after set-up
under overload	delay grows, packets are lost	new connections are refused
guarantees	none (“best effort”)	guaranteed rate
best suited to	bursty data traffic	steady, constant-rate traffic

Packets may be delayed, lost or reordered; repairing that, where needed, is the **hosts’** job → next week: the **end-to-end argument**.

Key idea No per-connection state in routers, no guarantees for packets: a deliberate division of labour between network and hosts.

6 · Why the access ISPs cannot all connect directly



7 ISPs → 21 links

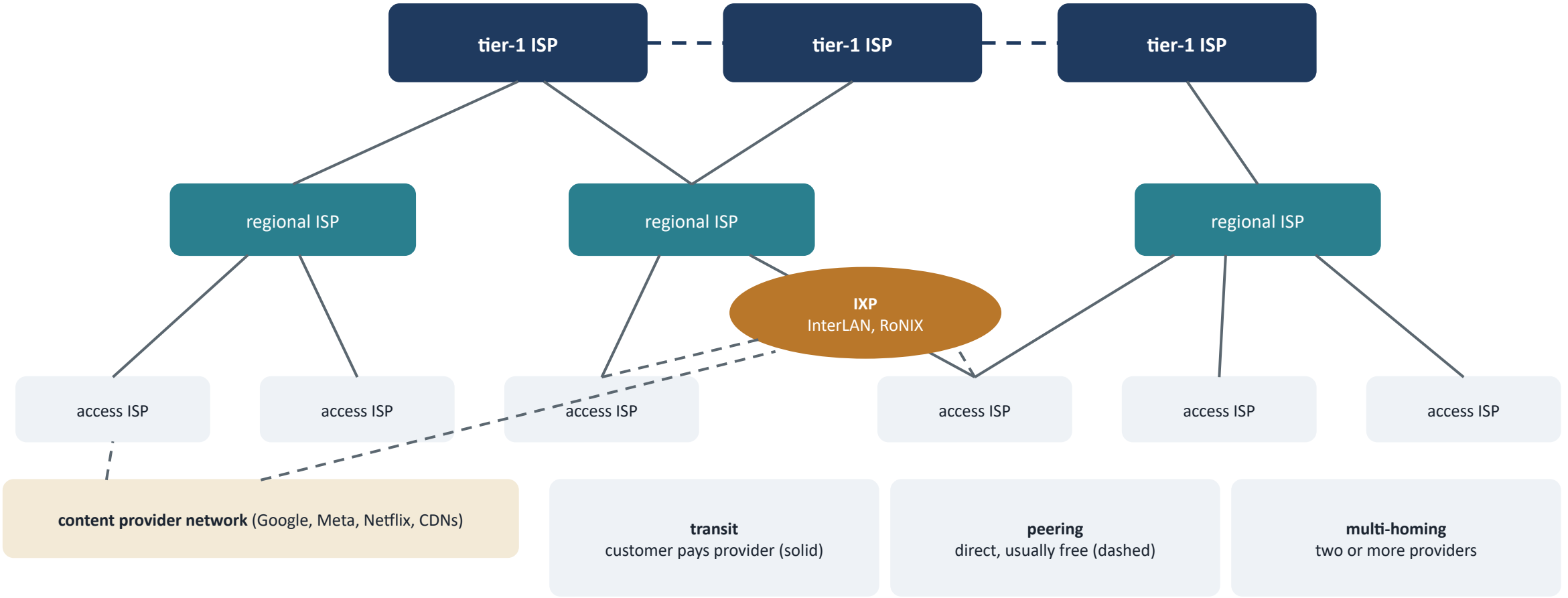
ISPs N	links N(N − 1)/2
7	21
100	4 950
1 000	≈ 500 000
100 000	≈ 5×10^9

links grow with the **square** of the number of networks
→ the real structure has levels, shaped by economics as much as engineering

Key idea Connecting every network to every other is impossible — the Internet is built as a hierarchy instead.

ISP (Internet service provider)

6 · A network of networks



Key idea A packet crosses several independently run networks; none controls the path — you will see it with traceroute.

RoNIX (Romanian Network for Internet eXchange) · ISP (Internet service provider) · IXP (Internet exchange point) · CDN (content delivery network)

6 · The six relationships, one by one

1 Transit

a small ISP pays a larger one to reach the rest of the Internet: customer → provider; regional ISPs buy from national, national from global

2 Tier-1 ISPs

about a dozen global networks that pay nobody for transit; they exchange traffic with each other without payment and so reach everything

3 Peering

two networks exchanging much traffic connect directly, usually without payment; both save a transit fee

4 IXP (Internet exchange point)

a facility — in essence a large switch — where many networks meet to peer; Romania: InterLAN and RoNIX (Romanian Network for Internet eXchange), Bucharest

5 Multi-homing

buying transit from two or more providers, to stay connected if one fails

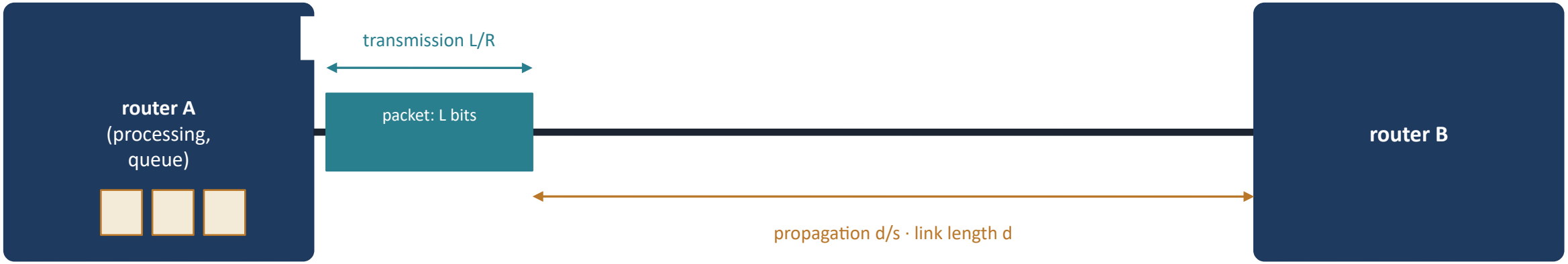
6 Content provider networks

Google, Meta, Netflix and CDNs (content delivery networks) run private global networks, peer with access ISPs and at IXPs, bypassing upper tiers

Key idea Who pays whom shapes the Internet's topology as much as cables do.

ISP (Internet service provider)

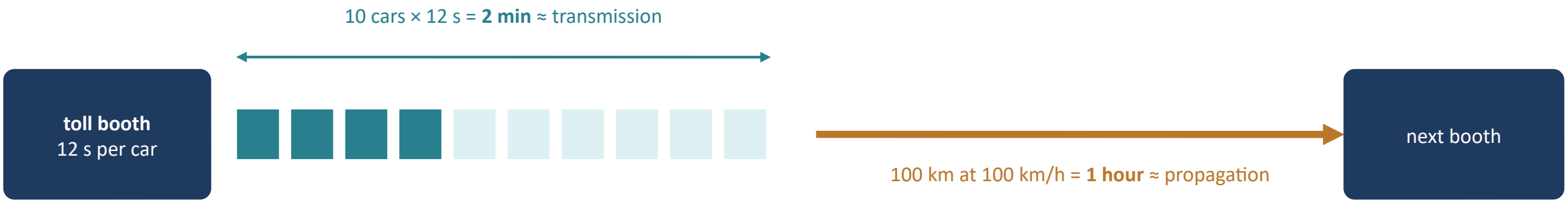
7 · Four sources of delay at one hop



1 · processing check header and bit errors, choose the output link	**2 · queueing** wait for the link to be free	**3 · transmission L / R** push all L bits onto a link of rate R	**4 · propagation d / s** one bit travels distance d at speed s ≈ 2×10^8 m/s
microseconds	0 … many ms, depends on congestion	depends on L and R — **not** on link length	depends on distance — **not** on L or R

Key idea d_nodal = d_proc + d_queue + d_trans + d_prop — and transmission is not propagation.

7 · Transmission is not propagation: the convoy



	convoy	packet
transmission	cars ÷ booth speed	L / R: bits ÷ link rate
propagation	distance ÷ driving speed	d / s: length ÷ signal speed

A faster toll booth does not shorten the drive. A shorter road does not speed up the booth.

Key idea Transmission depends on the packet and the rate; propagation on the distance — they never substitute for each other.

7 · Worked example 4: Cluj-Napoca to Bucharest



transmission, 100 Mbit/s

12 000 / 10^8 s = **0.12 ms**

propagation

450 000 / (2 × 10^8) s = **2.25 ms**

transmission, 1 Mbit/s

the same packet: **12 ms**

1 Here propagation dominates by a factor of almost 20; on a slow link the opposite is true.

Key idea Which delay dominates depends on the link: distance for fast long links, rate for slow ones.

7 · Throughput and the bottleneck



8 MB file = 64 × 10^6 bits → at 2 Mbit/s at least **64 / 2 = 32 s**, however fast the other links are

1 Delay tells how long one packet takes; throughput tells how fast a long transfer goes. Next week: delay, loss and throughput in depth.

Key idea A path is only as fast as its slowest link.

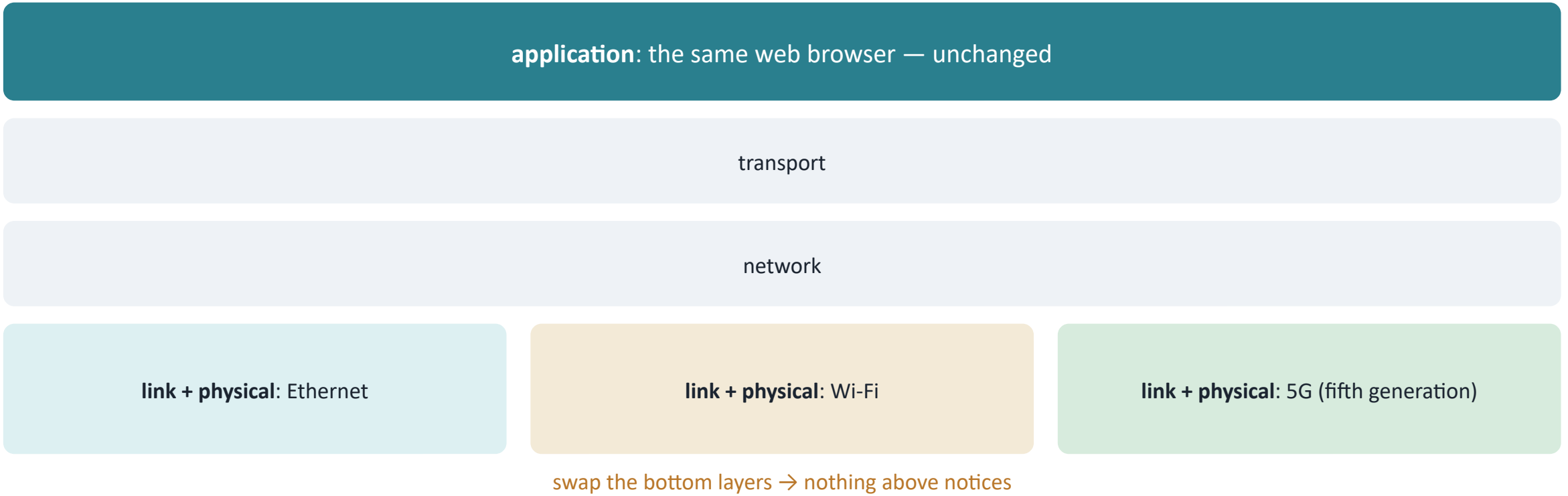
8 · Layering: five layers, each with one job

layer	its job	examples	unit
5 Application	gives meaning to the exchange between programs	HTTP, DNS, SMTP	message
4 Transport	data between *processes*; may add reliability and congestion control	TCP, UDP, QUIC	segment
3 Network	packets between *hosts* across many networks; addressing, routing	IP, ICMP; OSPF, BGP	datagram
2 Link	a packet across *one link* to the neighbour	Ethernet, Wi-Fi	frame
1 Physical	individual bits across the medium	fibre, copper, radio	bit

Key idea Bits, frames, datagrams, segments, messages: one unit of data per layer.

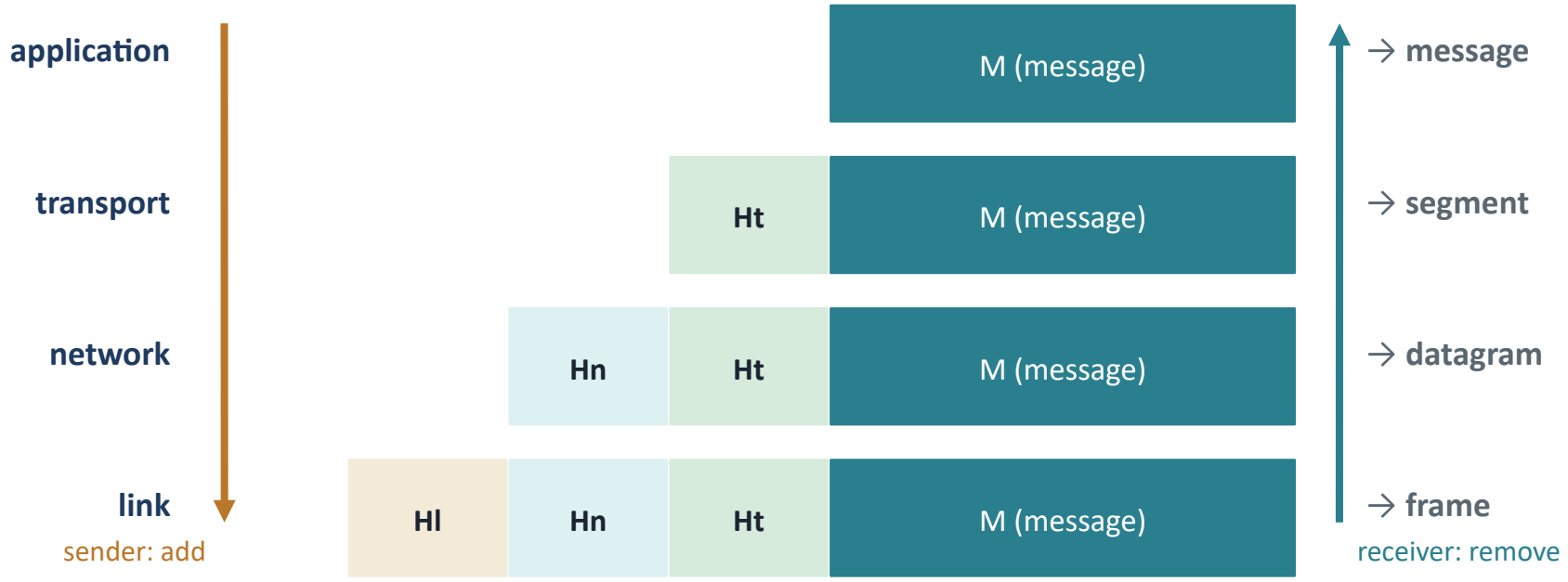
HTTP (HyperText Transfer Protocol) · SMTP (Simple Mail Transfer Protocol) · QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections) · ICMP (Internet Control Message Protocol) · OSPF (Open Shortest Path First) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · BGP (Border Gateway Protocol) · IP (Internet Protocol)

8 · Why layers: change one without touching the others



Key idea Layering divides one unmanageable problem into replaceable pieces with fixed boundaries.

8 · Encapsulation: each layer adds a header

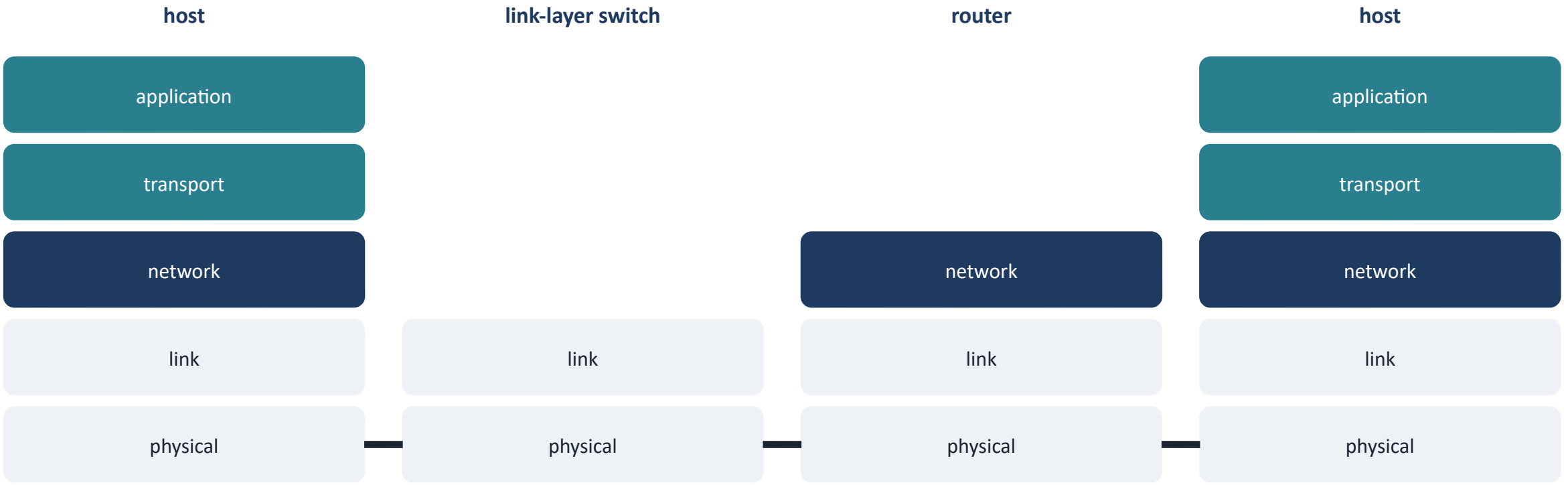


Ht = transport header · Hn = network header · Hl = link header

1 Each header carries what that layer's peer needs — ports, addresses, error checks.

Key idea Encapsulation: going down every layer wraps, going up every layer unwraps its own header.

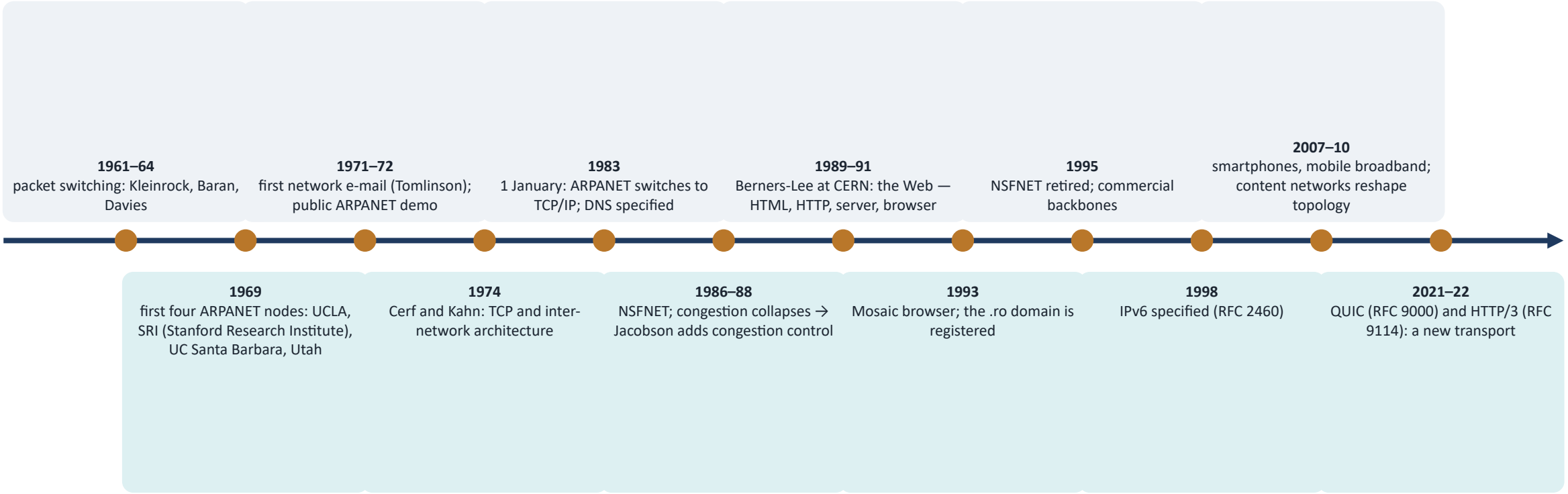
8 · Who implements which layers



5 layers · 2 layers · 3 layers · 5 layers — next week: this five-layer model versus the seven-layer OSI (Open Systems Interconnection) reference model

Key idea Applications and transport exist only at the edges; the core forwards and nothing more.

9 · A short history of the Internet



ARPANET (Advanced Research Projects Agency Network) · NSFNET (National Science Foundation Network) · CERN (Conseil Européen pour la Recherche Nucléaire) · UCLA (University of California, Los Angeles) · UC (University of California) · SRI (Stanford Research Institute)

Key idea An architecture from the 1970s still carries applications its designers never imagined.

HTTP (HyperText Transfer Protocol) · QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections) · HTML (HyperText Markup Language) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · DNS (Domain Name System) · RFC (Request for Comments) · IP (Internet Protocol)

Summary of Lecture 1

Network
hosts, links, packet switches; the Internet = network of networks on TCP/IP

Protocol
format, order, actions on send, receive and other events

Edge
access networks over guided or unguided media

Core
packet switching: store and forward, queues, loss; circuits reserve and waste

Forwarding vs routing
local table lookup vs network-wide computation of the tables

Network of networks
transit, peering, IXPs; content providers' own networks

Delay
processing, queueing, **L/R**, **d/s**; throughput = bottleneck

Layers
application, transport, network, link, physical; encapsulation

Key idea What the Internet is made of, how it moves data, how it is organised, and how long it takes.

TCP (Transmission Control Protocol) · IXP (Internet exchange point) · IP (Internet Protocol)

Review questions: where each answer is drawn

1. host vs packet switch; is a home Wi-Fi router a host?

→ 2.1 · nuts and bolts

2. three things a protocol defines, in the HTTP example

→ 3 · definition, real protocol

3. why residential access is asymmetric

→ 4.2 · two facts

4. half a packet received: why not forward yet?

→ 5.1 · store and forward

5. an application better served by circuits

→ 5.2 / 5.3 · circuits

6. forwarding vs routing

→ 5.1 · forwarding and routing

7. why two ISPs carry each other’s traffic free

→ 6 · peering

8. 100 Mbit/s → 1 Gbit/s: which delay changes?

→ 7 · four delays

9. which layers a router implements, and why

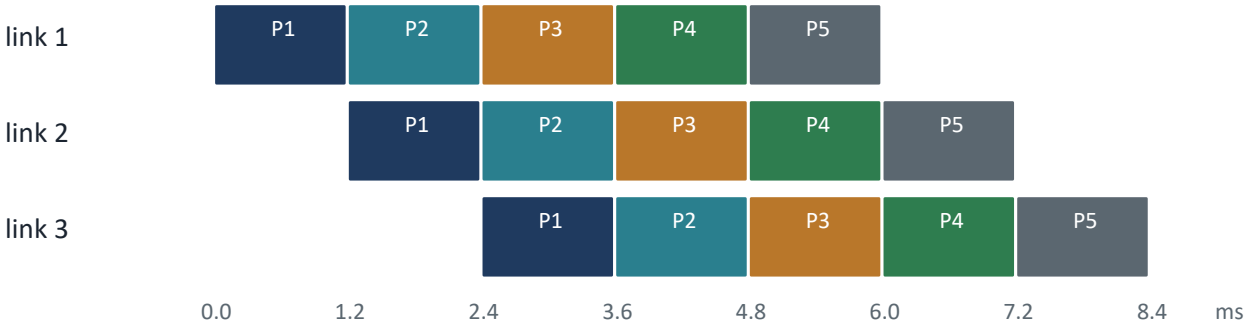
→ 8 · who implements which

Key idea Answer each question by redrawing its scheme from memory.

HTTP (HyperText Transfer Protocol) · ISP (Internet service provider)

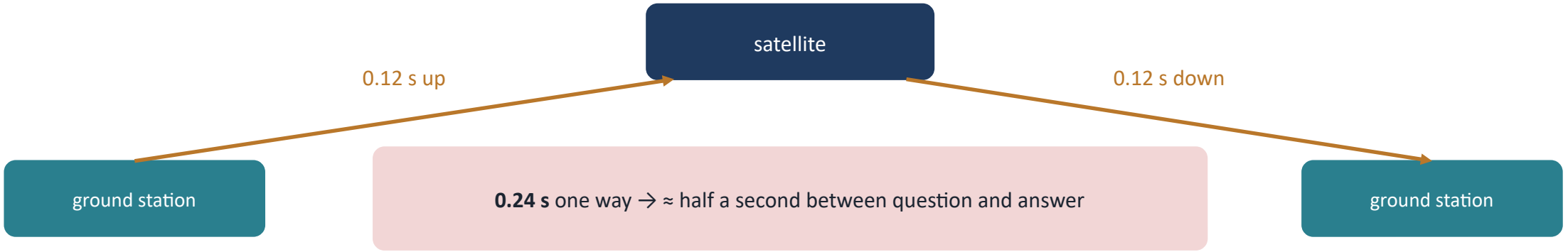
Exercises 1–2, solved as schemes

1 · 5 packets × 1500 B over 3 links of 10 Mbit/s: L/R = 1.2 ms



last bit arrives at (3 + 4) × 1.2 = **8.4 ms**
in general: (N + P − 1) · L/R

2 · geostationary satellite, 36 000 km up, radio at 3 × 10^8 m/s



0.24 s one way → ≈ half a second between question and answer

Key idea Pipelining hides all but one L/R per extra packet; distance cannot be hidden — low-orbit satellites avoid the half second.

Exercises 3–5, solved as schemes

3 · all 35 active at once

$$0.1^{35} = \mathbf{10^{-35}} \text{ — never, in practice}$$

The risk is not that everyone is active together but that **slightly more** users than the link carries are active: that is why **0.0004** (≥ 11 of 35) is the relevant figure.

4 · 40 TB, Iași → Bucharest

100 Mbit/s link



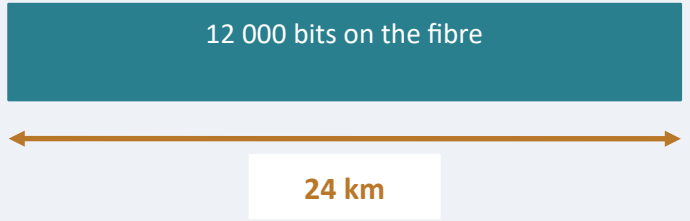
3.2×10^{14} bits / 10^{8} bit/s = 3.2×10^{6} s ≈ **37 days**

courier with disks

> **8 hours**

Courier: enormous throughput, terrible delay — two different measures.

5 · a packet in metres



transmission 0.12 ms; meanwhile the first bit travels 0.00012 s × 2 × 10^{8} m/s = **24 km**

Key idea Multiplexing risk lives just above capacity; throughput and delay are different measures; a packet has a physical length.

Reading, laboratory and next lecture

Required

Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., ch. 1: what the Internet is, edge, core, history

Recommended

Peterson & Davie, ch. 1 (free at book.systemsapproach.org) · Tanenbaum, Feamster & Wetherall, 6th ed., ch. 1

D. D. Clark, “The Design Philosophy of the DARPA Internet Protocols”, ACM SIGCOMM 1988

Laboratory 1

addresses and gateway with ip · round-trip times with ping · routers with traceroute

bring: how many networks does your packet cross? where does most of the delay come from?

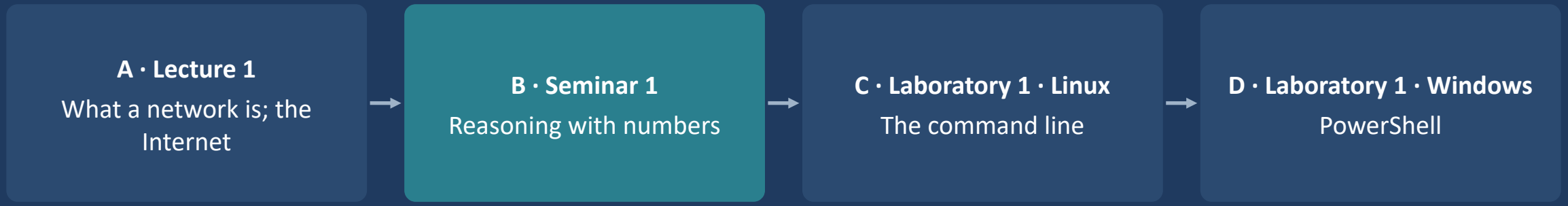
Next lecture

layered architectures: OSI and TCP/IP models, encapsulation step by step, delay, loss and throughput in depth

Key idea Read chapter 1, then find each of its parts on your own machine in Laboratory 1.

SIGCOMM (Special Interest Group on Data Communication) · DARPA (Defense Advanced Research Projects Agency) · TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · ACM (Association for Computing Machinery)

Part B · Seminar 1: reasoning with numbers



In this part

A · Warm-up: units and orders of magnitude

C · Problem 3: why packets?

E · Problem 6: design a protocol

Homework

B · Problems 1–2: delay on one link and on a path

D · Problems 4–5: sharing a link; queues

F · Problems 7–8: headers; reading a traceroute

A · Warm-up: one chain of conversions for the whole year



rates: decimal prefixes
1 kbit/s = 10^3 bit/s
1 Mbit/s = 10^6 · 1 Gbit/s = 10^9

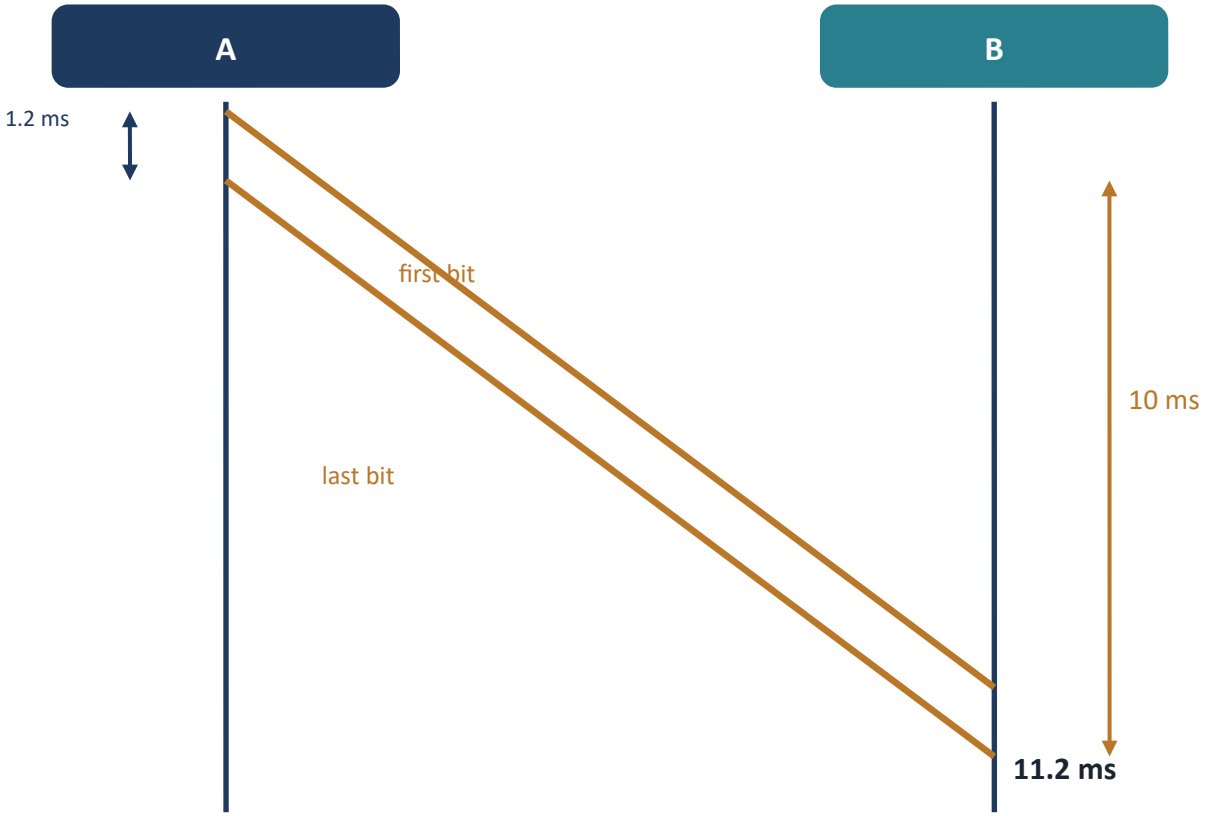
sizes: decimal here too
1 MB = 10^6 bytes
OS (operating system) tools often use 1 MiB = 2^{20} B (≈ 5 % more)

signal speed
copper, fibre: 2 × 10^8 m/s = **200 km per ms**
air, vacuum: 3 × 10^8 m/s

(a) 1500-byte packet, in bits?	(b) 5 MB at 20 Mbit/s?	(c) distance in fibre in 1 ms?	(d) “download: 4 MB/s” in Mbit/s?
12 000 bits	40 × 10^6 / 20 × 10^6 = **2 s**	**200 km**	**32 Mbit/s**

Key idea Bytes × 8 = bits; bits ÷ bit/s = seconds; 200 km per millisecond in fibre.

Problem 1 · One link: when does the last bit arrive?



1. d_trans = 12 000 / 10^7 s = **1.2 ms** · d_prop = 2 × 10^6 / 2 × 10^8 s = **10 ms**
2. last bit leaves at 1.2 ms, travels 10 ms → arrives at **11.2 ms**
3. when A finishes, the first bit is 1.2 ms × 200 km/ms = **240 km** away: the packet fills 240 km of the link
4. equal when d/s = L/R → d = s · L/R = **240 km**
5. one bit lasts 1/R = 0.1 µs → it occupies s/R = **20 m**

Key idea Draw time downwards: transmission is the gap between the two slanted lines, propagation is their slope.

Problem 2 · A path of three links: where does the time go?





trans 1 0.12 · prop 1 0.005 · proc 0.02

component	sum over the path	ms
transmission	0.12 + 1.2 + 0.012	1.332
propagation	0.005 + 2.5 + 0.25	2.755
processing	2 × 0.02	0.04

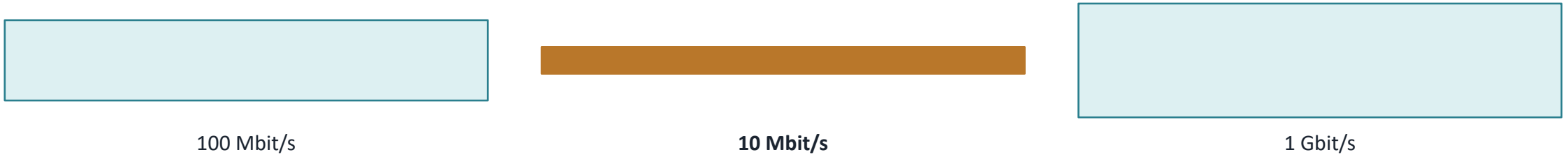
total
≈ 4.13 ms

Key idea The slow, long link dominates both propagation (2.5 ms) and transmission (1.2 ms).

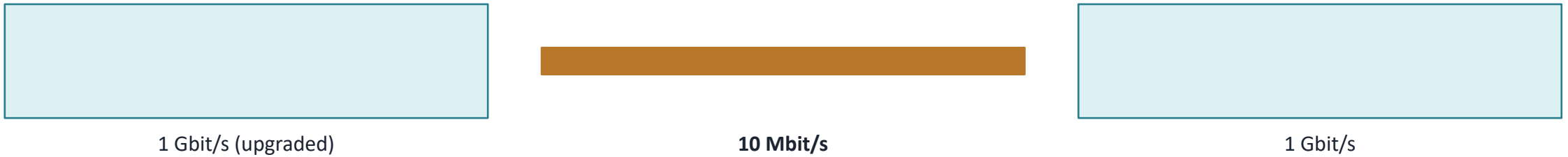
LAN (local area network)

Problem 2 · Q3–Q4: throughput, and an upgrade that changes nothing

before: bottleneck = 10 Mbit/s → 30 MB = 240 × 10^6 bits → at least **24 s**



after upgrading link 1: bottleneck still 10 Mbit/s → still **24 s**



Upgrading a link that is not the bottleneck does **not** raise throughput — find the bottleneck first.

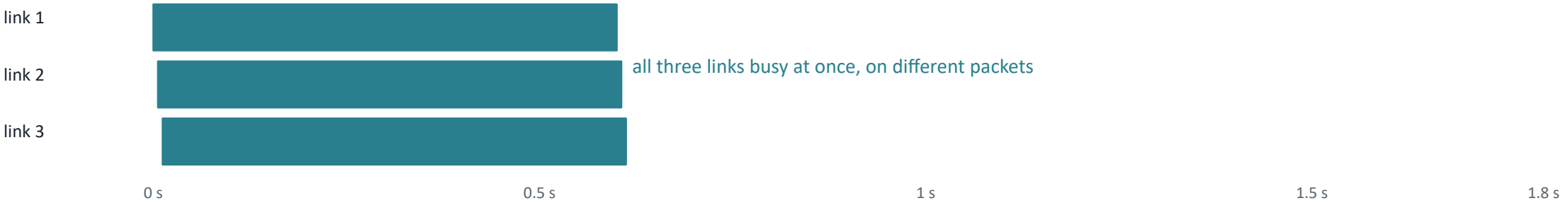
Key idea Throughput = the slowest link; money spent anywhere else is wasted for long transfers.

Problem 3 · Message switching vs packet switching

message switching — the whole message, hop after hop: 3 × 0.6 s = **1.8 s**



packet switching — 100 packets of 12 000 bits, L/R = 6 ms: 102 × 6 ms = **0.612 s**



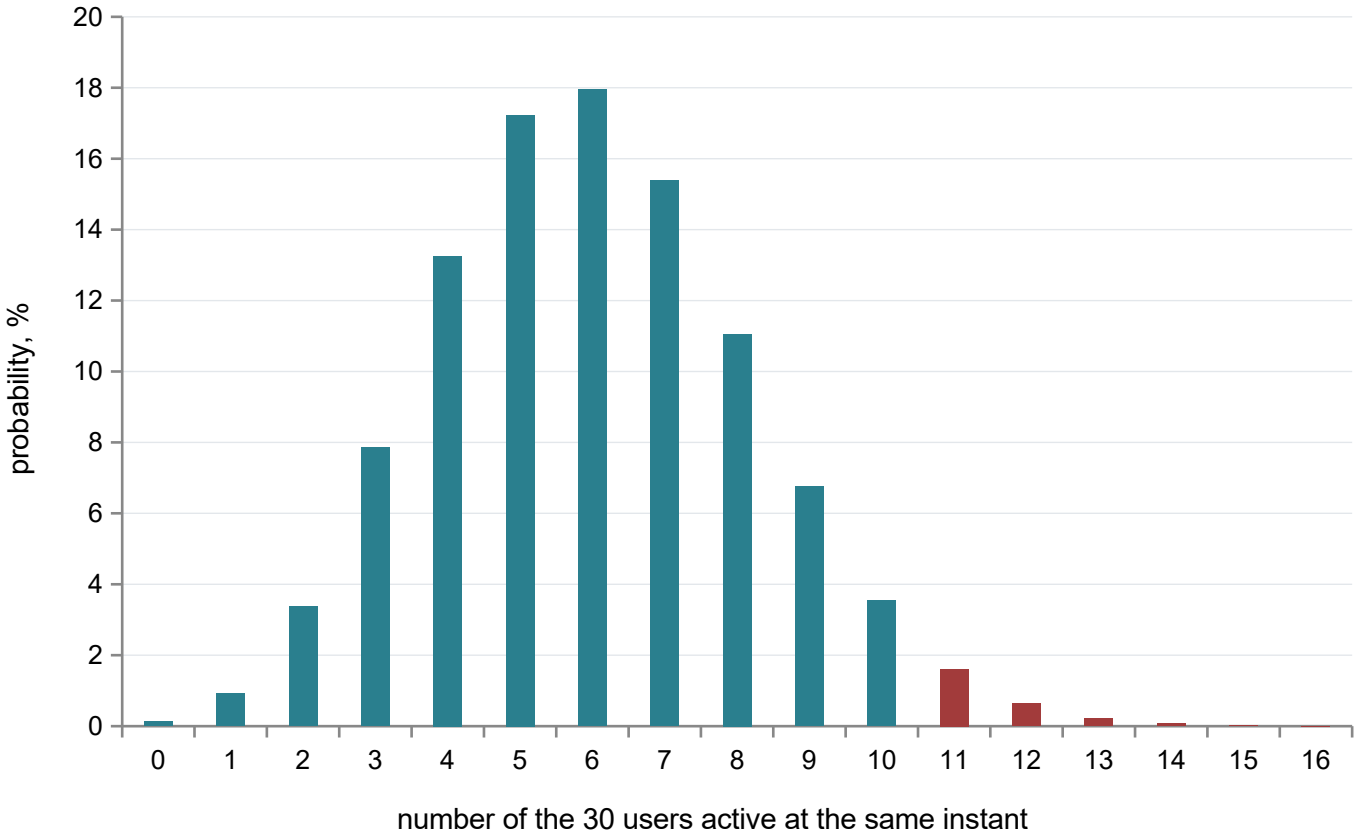
+ 40-byte headers: 12 320 bits
102 × 6.16 ms = **0.628 s**, overhead **2.6 %**

advantages: an error resends one small packet;
a long message does not block a switch for everyone

disadvantage: header overhead;
reordering and reassembly

Key idea Cutting messages into packets lets every link work in parallel: almost three times faster here.

Problem 4 · Circuits or packets? 30 bursty users on 100 Mbit/s



1 · circuits: 100 / 10 = **10 users**

2 · average: 30 × 0.2 = **6 active**
60 Mbit/s = 60 % of capacity

3 · overload = more than 10 active (red bars)
$\Sigma\ C(30,k)\ 0.2^{k}\ 0.8^{30-k}$, k = 11…30 ≈ **0.026**
≈ 2.6 % of the time demand exceeds capacity and packets queue

Key idea Three times as many users as circuits allow, at the price of queueing about 2.6 % of the time.

Problem 4 · Q4: would you admit 30 users?

web browsing
short queueing episodes 2.6 % of the time are acceptable
→ admit: three times as many customers

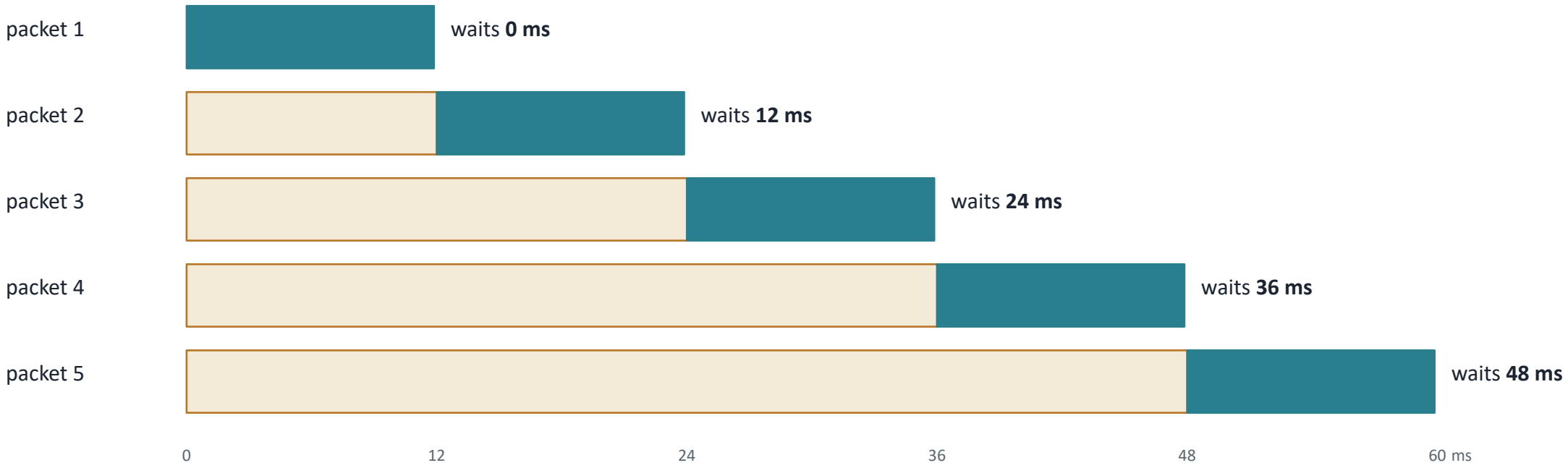
voice call · remote surgery
even brief queueing may be unacceptable
→ maybe not, or reserve capacity

Compare with the lecture example (probability that demand exceeds the link):



Key idea Fewer but busier users overload a link 65 times more often: burstiness matters more than numbers.

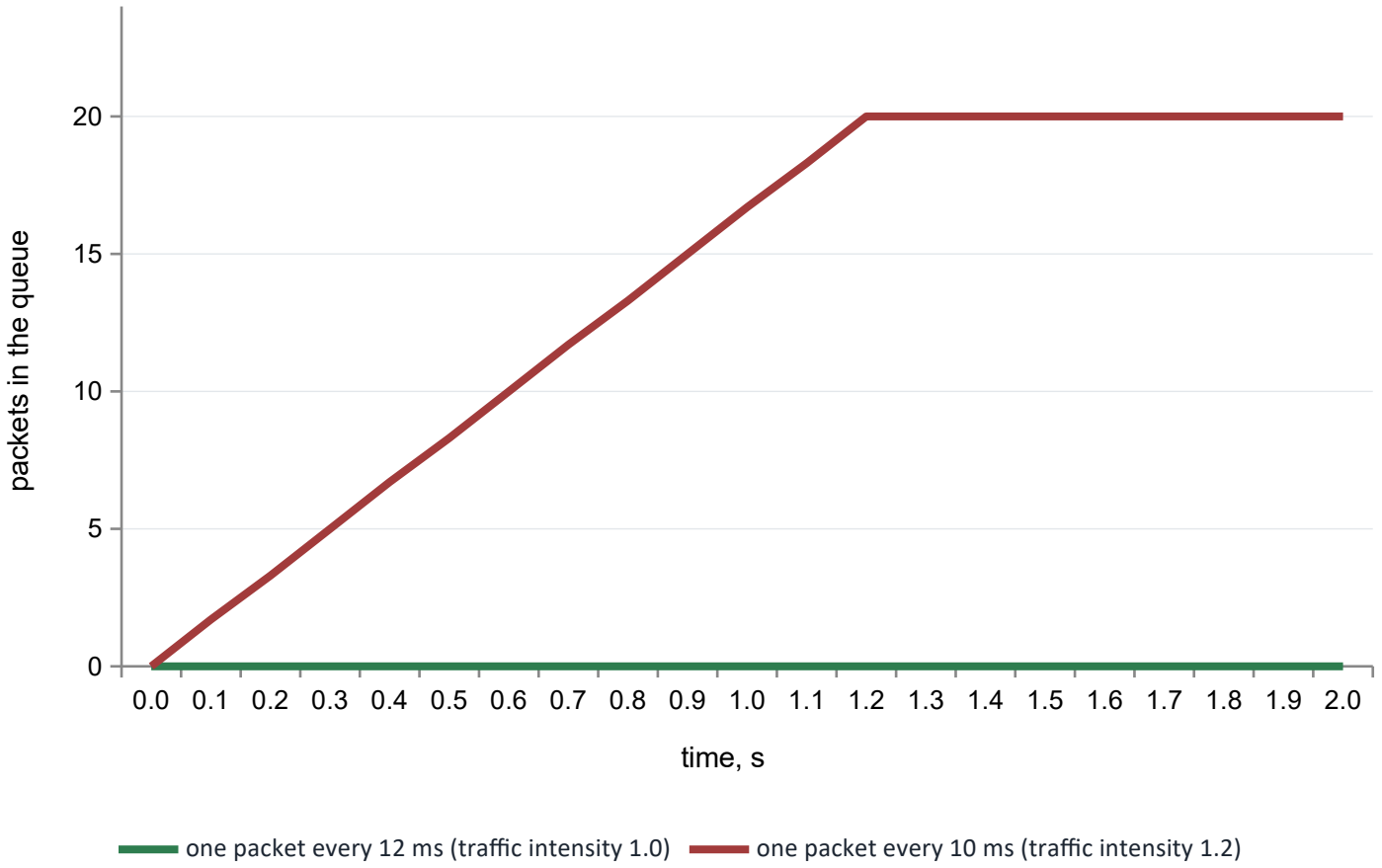
Problem 5 · A queue forms: five packets at once



the n-th packet waits **(n − 1) · L/R**
average: (0 + 12 + 24 + 36 + 48) / 5 = **24 ms**

Key idea Simultaneous arrivals queue up even on an idle link: the last one pays for all the others.

Problem 5 · Q2–Q4: arrivals every 12 ms vs every 10 ms



12 ms: each packet finishes as the next arrives
queueing delay **0** · ratio = **1.0**

10 ms: 100 packets/s arrive, 83.3 can leave
+16.7 packets/s → 20-packet buffer full after ≈ **1.2 s** → ≈ 1 in 6 dropped · ratio **1.2**

traffic intensity = arrival rate (bit/s) / link rate
above 1 the queue grows without limit; a bigger buffer only delays the loss and adds delay

Key idea Traffic intensity 1 is the edge of the cliff: just above it, no buffer is big enough. More next week.

Problem 6 · Design a time protocol

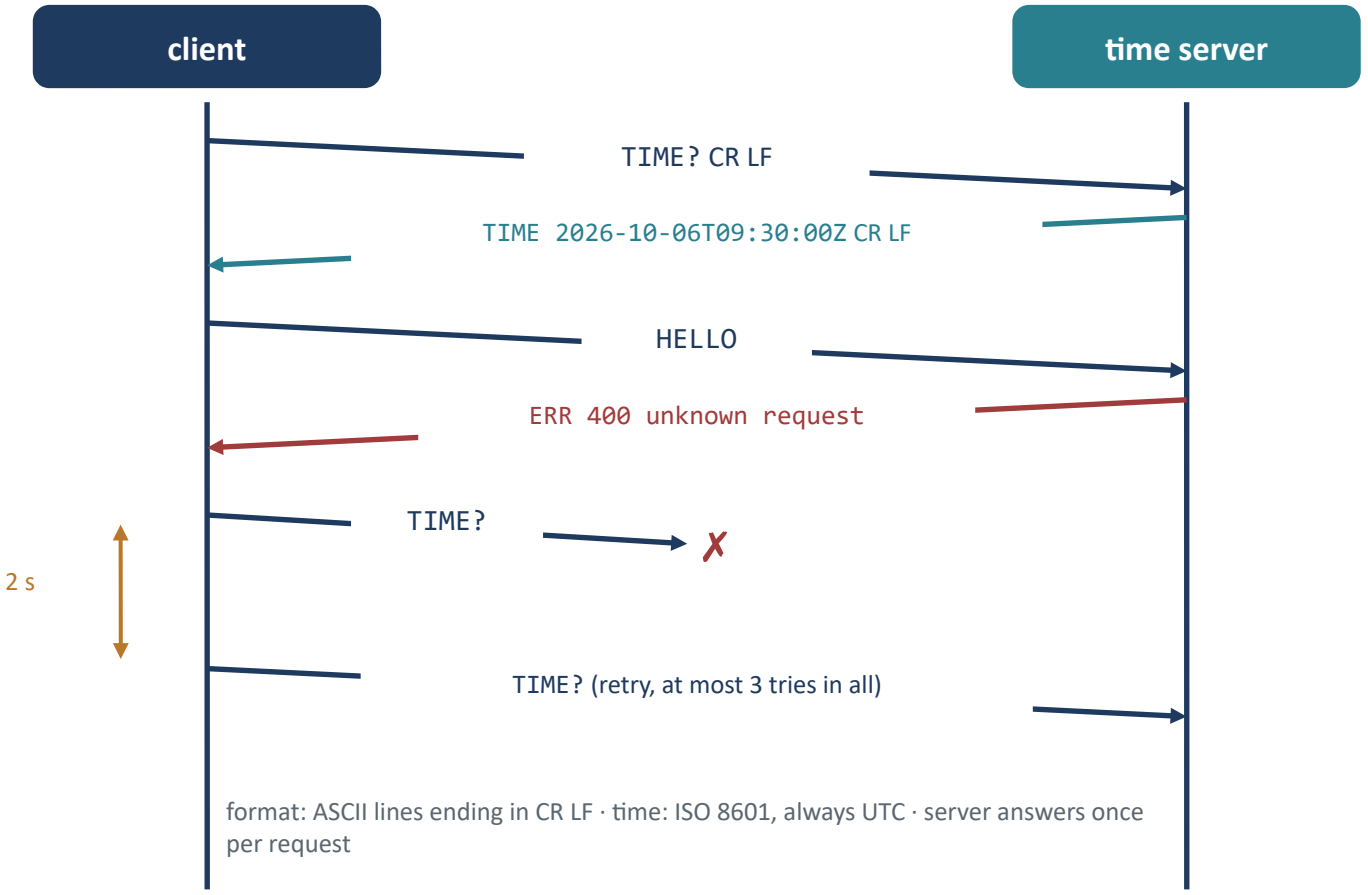
What exactly does the client send? Text or binary? Where does the message end?

What exactly does the server reply? Format and time zone?

What does the server do with a message it does not understand?

What does the client do if no reply comes? After how long? How many tries?

Is it harmful if the server receives the same request twice?



Key idea Format, order, actions, errors and timeouts: a complete specification answers every question before a program meets it.

ASCII (American Standard Code for Information Interchange) · UTC (Coordinated Universal Time) · ISO (International Organization for Standardization) · CR (carriage return) · LF (line feed)

Problem 6 · Repeating a request: idempotent or not?

idempotent: TIME?

TIME? ×2 → server state unchanged

duplicates are harmless → the client may simply resend

not idempotent: “transfer 100 lei”

request **ID 731**
transfer 100 → server remembers ID
731 seen → do not repeat

the identifier lets the server recognise a repetition — transport-layer lectures

every group forgot something
usually the end-of-message rule or the timeout

real RFCs are long
for exactly this reason

the lecture’s definition is a checklist
format · order · actions

Key idea Before a client may resend, the protocol must say whether a repetition is harmless — or how to detect it.

RFC (Request for Comments)

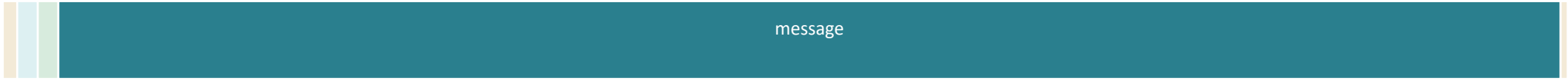
Problem 7 · The cost of headers

100-byte message → 158-byte frame, overhead **58 / 158 ≈ 37 %**



(drawn 6 × larger than the bar below)

1460-byte message → 1518-byte frame, overhead **58 / 1518 ≈ 3.8 %**

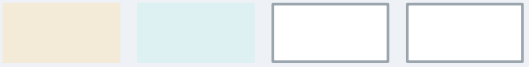


Q3 · who reads which header?

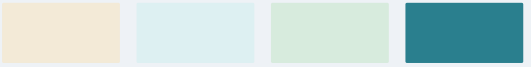
link-layer switch
Ethernet header only



router
Ethernet + IP headers



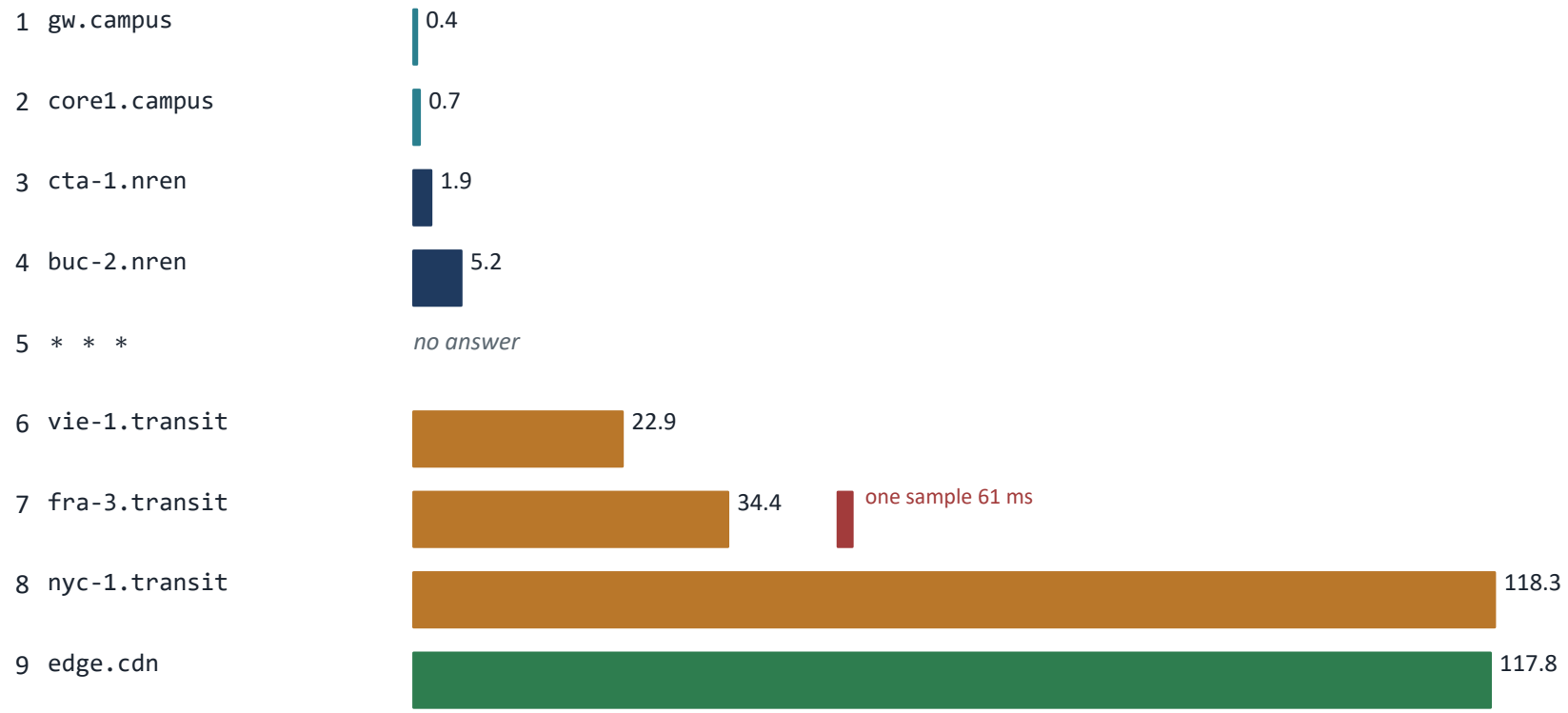
destination host
all headers; its application reads the message



Key idea Small messages are expensive — one reason applications batch data.

TCP (Transmission Control Protocol) · IP (Internet Protocol)

Problem 8 · Reading a traceroute: four networks, one ocean





Q1 · four networks
domain names and address blocks change at each boundary

Q2 · Atlantic: hops 7 → 8
+84 ms RTT → ≈ 42 ms one way → ≤ 8400 km of fibre

Key idea Network boundaries show in the names; geography shows in the jumps — Frankfurt to New York here.

Problem 8 · Q3–Q5: what the odd lines mean

Q3 · * * * at hop 5

the router did not answer the probes — many are configured not to, or limit how often they do

not broken: later hops answer

Q4 · 61 ms at hop 7

a momentary queue on the path, or the router slow to generate the reply: answering probes is low-priority work

one slow sample of three is **noise**

Q5 · hop 9 < hop 8

each line uses different packets at a different moment; reply-generation time varies

only **large, consistent** differences mean something

Key idea Read traceroute statistically: trust consistent patterns, not single numbers.

Homework

1 Exercises 1–5

at the end of the lecture notes — solve them without looking at the answers

2 Problem 4 again

users active **50 %** of the time, 15 users: is statistical multiplexing still worthwhile?

3 RFC 867

the one-page “Daytime Protocol”: compare it with your Problem 6 design — what did its author leave unspecified?

Key idea Busier users and a real one-page protocol: test the seminar’s two big ideas on new cases.

RFC (Request for Comments)

Part C · Laboratory 1 on Linux

A · Lecture 1
What a network is; the Internet

→

B · Seminar 1
Reasoning with numbers

→

C · Laboratory 1 · Linux
The command line

→

D · Laboratory 1 · Windows
PowerShell

In this part

Before you start · plan

2 · The way out: routes, neighbours, DNS

4 · The path: a network of networks

6 · A protocol by hand

8 · Seeing packets (optional)

1 · Who am I? Interfaces and addresses

3 · Reachability and round-trip time

5 · Names and addresses

7 · Throughput

If something does not work

DNS (Domain Name System)

Before you start (Linux)

```
sudo apt update
sudo apt install -y iproute2 iputils-ping traceroute mtr-tiny dnsutils curl netcat-openbsd tcpdump
```

Environment
course VM (virtual machine) · native Linux · WSL (Windows Subsystem for Linux) 2

Getting help
`man ping` · `ping --help` · `ip addr help` — reading the manual is part of the work

Rules of conduct
probe only the lab network and well-known public servers, with the small packet counts given; no scanning of address ranges or ports

Hand in
one file (text or PDF, Portable Document Format) with answers to **Q1–Q16** and the output supporting each; outputs shown here are illustrations

Key idea One install line, the manual pages, and a clear rule: probe only what you are allowed to probe.

mtr (my traceroute)

Laboratory 1 (Linux) · the two hours



1 Who am I? Interfaces and addresses

2 The way out: routes, neighbours, DNS (Domain Name System) servers

3 Reachability and round-trip time

4 The path: a network of networks

5 Names and addresses

6 A protocol by hand

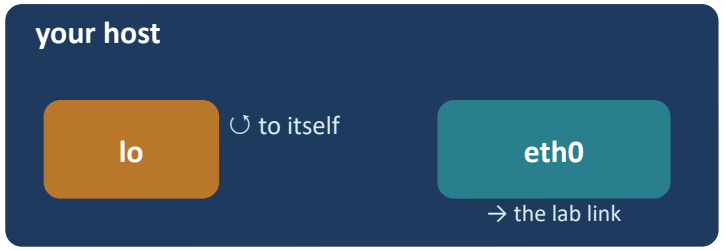
7 Throughput

8 Seeing packets (optional) and wrap-up

Key idea Tasks 4 and 6 are the heart of the lab: the network of networks, and a protocol typed by hand.

Task 1 · Who am I? Interfaces and addresses

```
$ ip -br addr
lo       UNKNOWN  127.0.0.1/8 ::1/128
eth0     UP       10.20.3.57/24 fe80::5054:ff:fe12:3456/64
```



`10.20.3.57`	`/24`	`link/ether 52:54:00:12:34:56`	`mtu 1500`
IPv4 address network layer	**prefix length** network part = first 24 bits	**MAC (Media Access Control) address** link layer	**MTU (maximum transmission unit)** largest packet the link carries

the last two come from the full `ip addr show`

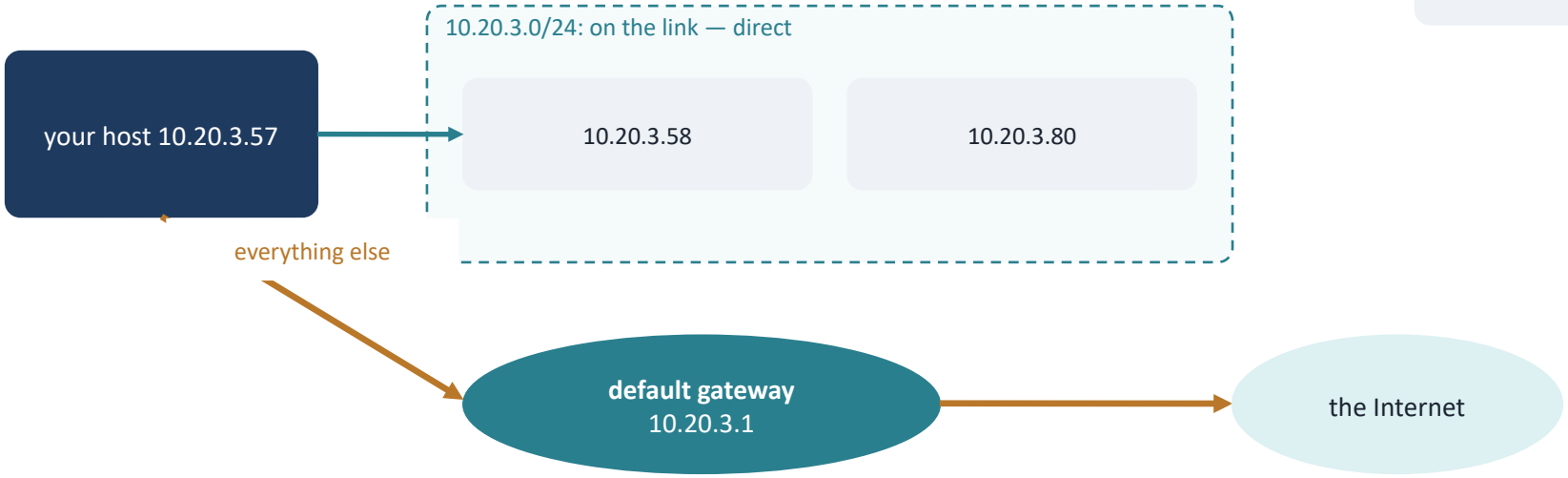
Q1	list your interfaces; which one connects you to the lab — wired or wireless?
Q2	its IPv4 address and prefix, MAC address, MTU; which **layer** does each address belong to?
Q3	compare with a neighbour's address: which part is the same? (the network prefix)

Key idea Two addresses per interface: the IP address names the host in the network layer, the MAC address the card on the link.

IPv4 (Internet Protocol version 4) · IP (Internet Protocol)

Task 2 · The way out: the host's forwarding table

```
$ ip route
default via 10.20.3.1 dev eth0 proto dhcp metric 100
10.20.3.0/24 dev eth0 proto kernel scope link src 10.20.3.57
```



`ip route get 1.1.1.1` → which route?
`ip neigh` → neighbours and their MAC addresses
`resolvectl status`, `/etc/resolv.conf` → DNS server

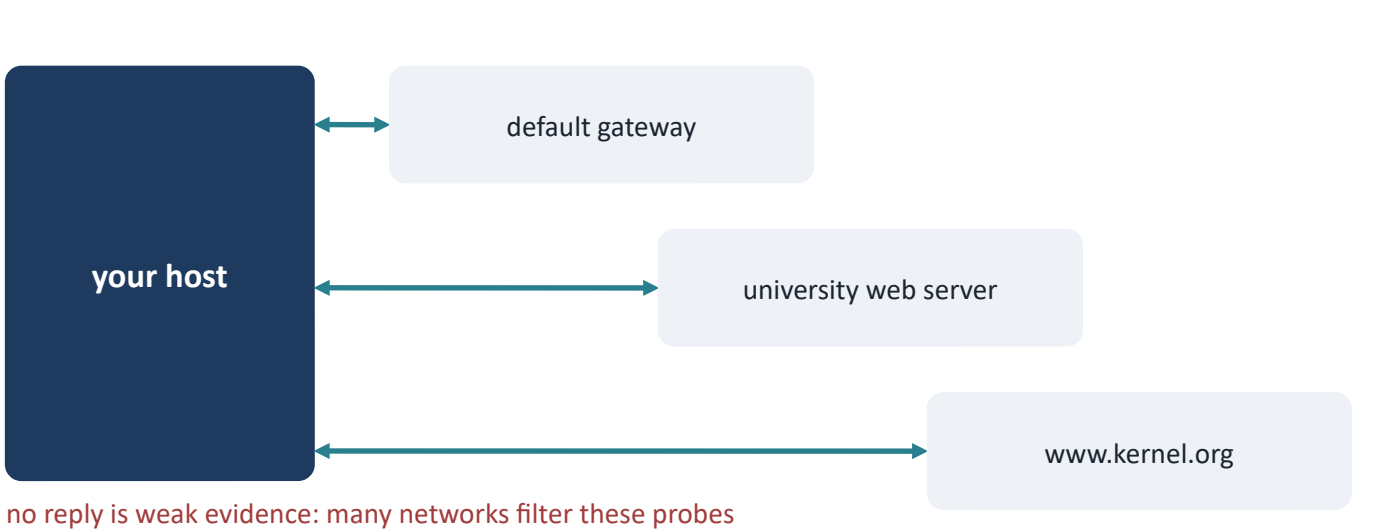
Q4 · gateway address; **2** entries here vs **hundreds of thousands** in a core router — a host knows one way out

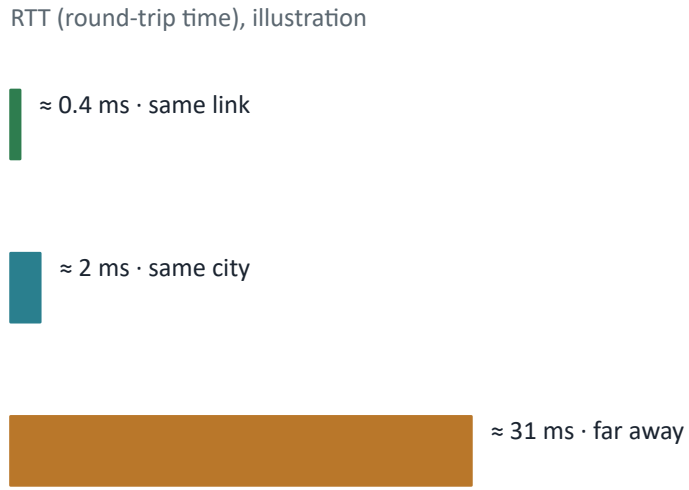
Q5 · which DNS (Domain Name System) server? is it in your own network?

Key idea A host's table has two rules: "on my link, deliver directly; anything else, to the gateway".

MAC (Media Access Control)

Task 3 · Reachability and round-trip time with ping





no reply is weak evidence: many networks filter these probes

Q6 min / avg / max RTT and loss for each: most of the difference is **propagation**

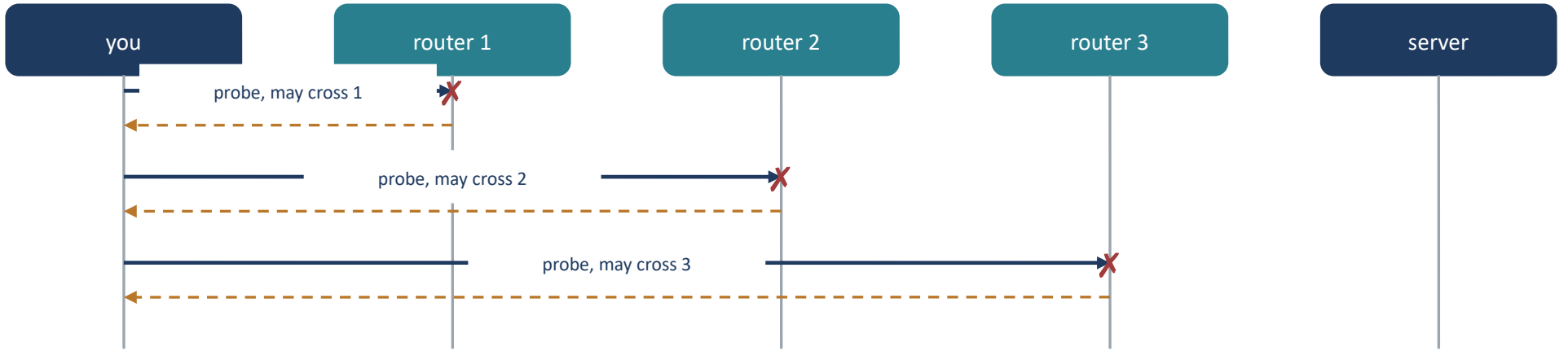
Q7 min RTT 31 ms → one way ≤ 15.5 ms × 200 km/ms ≈ **3100 km**: an upper bound (path not straight, other delays); big sites serve from many places

Q8 `-s 1400` (larger packets): RTT rises only slightly — **transmission** delay depends on packet size

Q9 TTL (time to live) starts at 64, 128 or 255 and drops by one per router: reply ttl 52 → 64 − 52 = **12 routers**

Key idea Gateway, city, other side of Europe: RTT grows with distance, because propagation dominates.

Task 4 · The path: traceroute and mtr



`traceroute -n` addresses only

`sudo traceroute -I` ping-style probes

`mtr -rwc 20`: loss and delay per hop

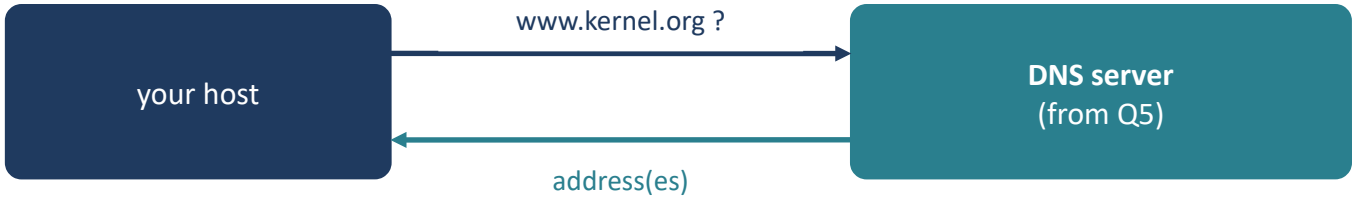
you | **campus** hops 1–2 | **research network or ISP** hops 3–4 | **transit provider(s)** hops 6–8 | **destination network** hop 9

Q10	number of hops; agrees with the estimate from TTL (time to live) in Q9?	**Q11**	mark where the path leaves each network (names, address blocks); draw it like Figure 1 of the lecture
Q12	the largest RTT jump: probably a geographic step (a border, a sea)	**Q13**	a Romanian and a far server: the paths share the first hops — the same way out of campus

Key idea Asterisks and later hops faster than earlier ones are normal — see seminar Problem 8.

ISP (Internet service provider) · RTT (round-trip time) · mtr (my traceroute)

Task 5 · Names and addresses with dig




```
;; ANSWER SECTION:
www.kernel.org.  300  IN  A  …
;; Query time: 23 msec
```

`dig +short www.kernel.org`
IPv4 address(es): several?

`dig +short AAAA www.kernel.org`
AAAA = IPv6 address record

`dig -x <router address>`
reverse: address → name

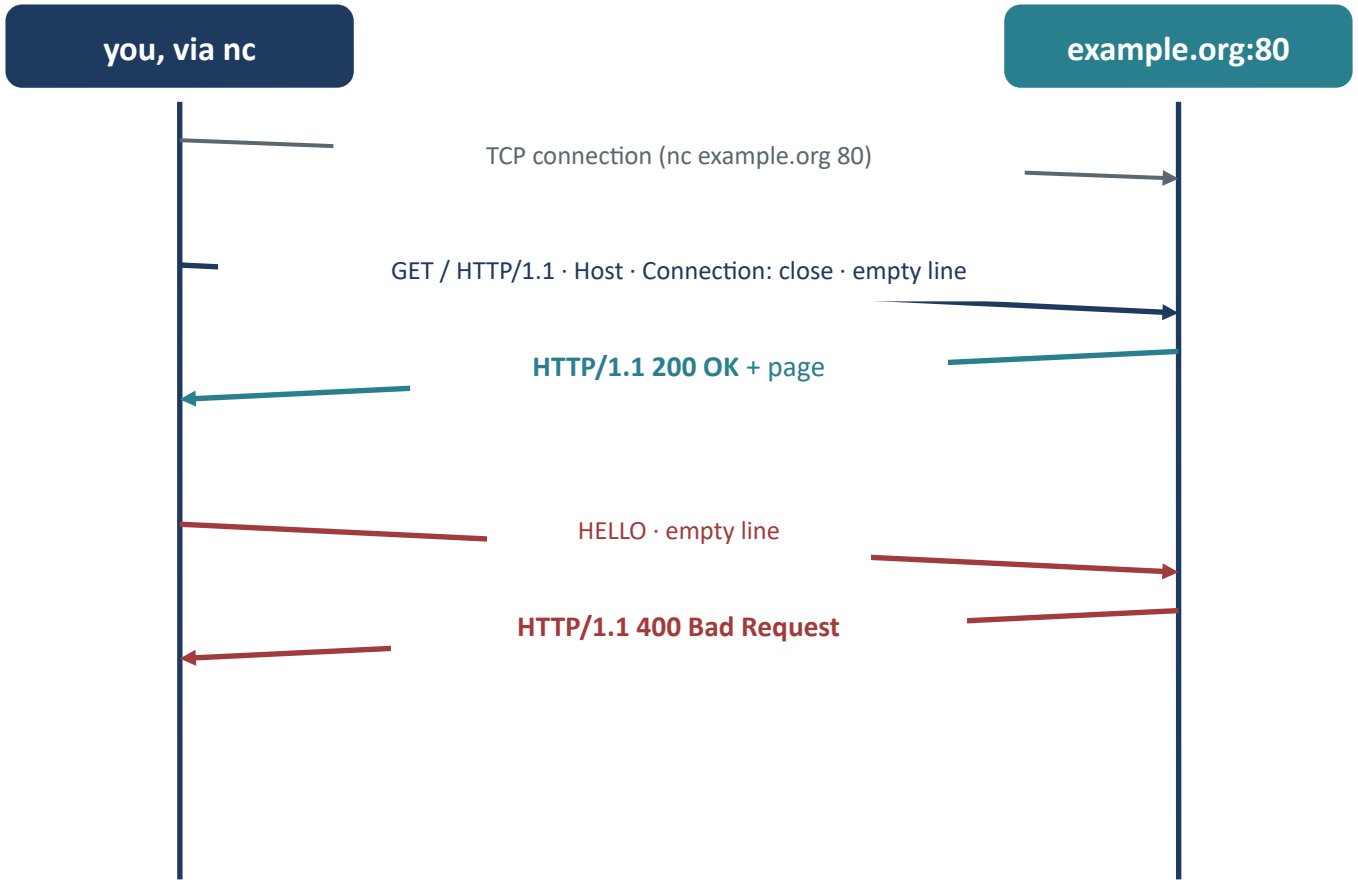


Q14 · the second query is fast:
the answer is **cached** by the resolver

Key idea Names are for people, addresses for packets; DNS answers once and then serves the cached copy.

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · DNS (Domain Name System) · dig (domain information groper) · IN (Internet class)

Task 6 · A protocol by hand: be the browser




```
printf 'GET / HTTP/1.1\r\nHost: example.org
\r\nConnection: close\r\n\r\n' | nc example.org 80
```

```
$ ss -tn
ESTAB 0 0 10.20.3.57:51234 93.184.215.14:80
```

while connected: the connection seen from the host

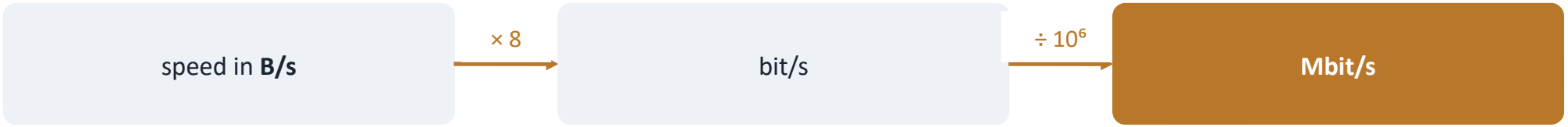
Q15 · first reply line to GET and to HELLO
format (request line, headers, empty line) · order (client first, one reply) · actions (wrong format → **400** error)

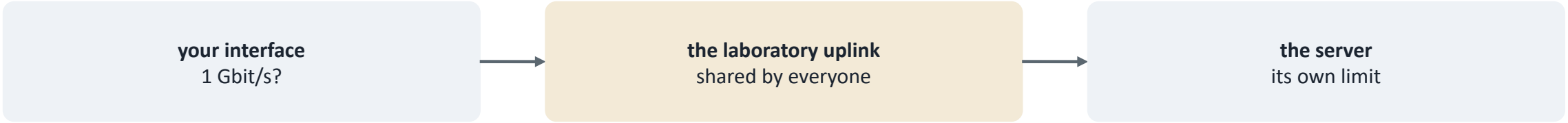
Key idea A protocol is just agreed text on a connection — break the format and the server says so (400 Bad Request; the exact status may vary).

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · ss (socket statistics) · nc (netcat)

Task 7 · Throughput

```
curl -o /dev/null -w 'speed: %{speed_download} B/s   time: %{time_total} s\n' <test file URL>
```





Q16 · where is the bottleneck? Evidence: if the rate **falls when colleagues download together**, the shared lab uplink is the bottleneck; if it stays at the same value well below your interface speed, suspect the server.

Key idea Measure, then disturb the system: what changes the rate tells you where the bottleneck is.

Task 8 (optional) · Seeing packets with tcpdump

```
$ sudo tcpdump -i eth0 -n -e -c 5 icmp
09:30:01.1 52:54:00:12:34:56 > 52:54:00:aa:bb:01, IPv4: 10.20.3.57 > 10.20.3.1: ICMP echo request
09:30:01.1 52:54:00:aa:bb:01 > 52:54:00:12:34:56, IPv4: 10.20.3.1 > 10.20.3.57: ICMP echo reply
```

link layer: MAC header (shown with -e)

network layer: IP header 10.20.3.57 > 10.20.3.1

ICMP (Internet Control Message Protocol) echo request — the ping probe

1 One line = one packet: time, source, destination, type. Next week: Wireshark opens every header.

Key idea tcpdump -e shows the frame around the packet around the message: Figure 3 of the lecture, live.

IPv4 (Internet Protocol version 4) · MAC (Media Access Control) · IP (Internet Protocol)

If something does not work (Linux)

`command not found`	the package is not installed — see “Before you start”
ping works to the gateway, not beyond	routing or filtering beyond your link: check `ip route`; try another destination
ping to an address works, to a name does not	name resolution fails: check `/etc/resolv.conf`; try `dig @1.1.1.1 example.org`
traceroute shows only asterisks	default probes filtered: `sudo traceroute -I` or `sudo traceroute -T -p 443`
`Operation not permitted`	needs administrator rights: prefix with `sudo`

Key idea Work outwards: your link, your gateway, names, then the path beyond.

dig (domain information groper)

Part D · Laboratory 1 on Windows

A · Lecture 1
What a network is; the Internet

→

B · Seminar 1
Reasoning with numbers

→

C · Laboratory 1 · Linux
The command line

→

D · Laboratory 1 · Windows
PowerShell

In this part

Before you start: two families of tools

1 · Who am I? Adapters and addresses

2 · The way out: routes, neighbours, DNS

3 · Reachability and round-trip time

4 · The path: tracert and pathping

5 · Names, addresses and the DNS cache

6 · Ports, connections, a protocol by hand

7 · Throughput

8 · Linux and Windows side by side

If something does not work

DNS (Domain Name System)

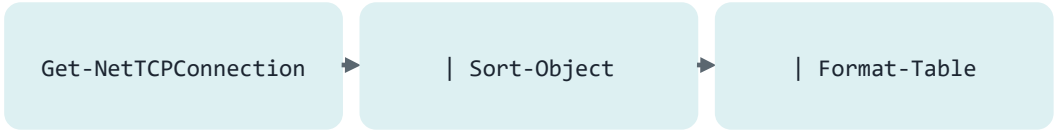
Before you start (Windows): two families of tools

classic commands → text

```
ipconfig · ping · tracert · netstat
→ lines of text to read
```

(admin) marks commands needing “Run as administrator”

PowerShell cmdlets → objects



objects flow through a pipeline: filter, sort, select columns

A trap
in Windows PowerShell 5.1 `curl` is an alias for another command: always type `curl.exe`

Getting help
`Get-Help Test-NetConnection -Examples` · `ping /?`

Hand in
answers to **Q1–Q16** with output; append with `| Out-File report.txt -Append`

Key idea Use both: classic commands to read quickly, cmdlets to filter and sort.

Laboratory 1 (Windows) · the two hours



1 Who am I? Adapters and addresses

2 The way out: routes, neighbours, DNS (Domain Name System) servers

3 Reachability and round-trip time

4 The path: a network of networks

5 Names and addresses

6 Ports, connections and a protocol by hand

7 Throughput

8 Linux and Windows side by side; wrap-up

Key idea Same concepts, different tool names — the plan mirrors the Linux sheet task by task.

Task 1 · Who am I? Adapters and addresses

```
Ethernet adapter Ethernet:
   Physical Address. . . : 00-1A-2B-3C-4D-5E
   DHCP Enabled. . . . . : Yes
   IPv4 Address. . . . . : 10.20.3.58(Preferred)
   Subnet Mask . . . . . : 255.255.255.0
   Default Gateway . . . : 10.20.3.1
   DNS Servers . . . . . : 10.20.0.53
```

- Physical Address = **MAC** (Media Access Control) address — *link layer*
- DHCP (Dynamic Host Configuration Protocol): address received automatically — *Q3*
- IPv4 address — *network layer*
- 255.255.255.0 = **/24** — *24 one-bits*
- the edge router — *Task 2*
- name server — *Task 5*



Q1	adapters; which one is **Up**, and its link speed (`Get-NetAdapter`)
Q2	IPv4, mask and prefix, MAC; the layer of each address
Q3	typed by hand or DHCP? from which server, until when (Lease Expires)?

Key idea Same two addresses as on Linux — only the names change: Physical Address, Subnet Mask.

IPv4 (Internet Protocol version 4) · DNS (Domain Name System)

Task 2 · The way out: routes, neighbours, DNS servers

```
Network Destination    Netmask         Gateway      Interface
        0.0.0.0        0.0.0.0      10.20.3.1     10.20.3.58
       10.20.3.0  255.255.255.0      On-link     10.20.3.58
```

0.0.0.0 / 0.0.0.0 matches everything
= the default route → gateway

`Find-NetRoute -RemoteIPAddress 1.1.1.1`
which route and which source address would be used?

`arp -a` · `Get-NetNeighbor`
neighbours on the link and their MAC addresses

`Get-DnsClientServerAddress`
which DNS servers the host uses

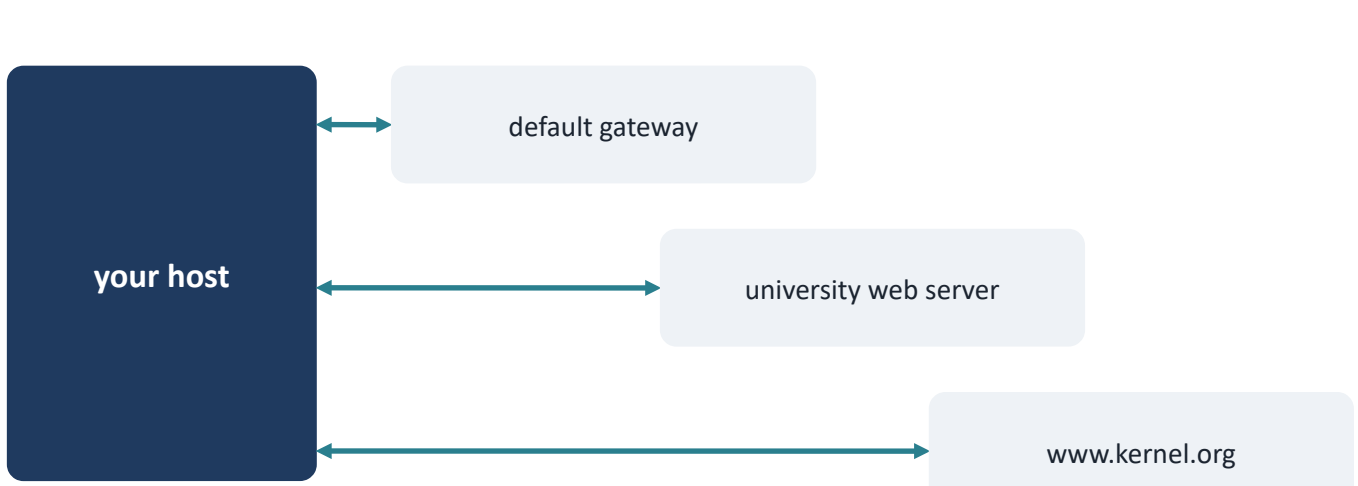
Q4 · gateway address, then its **MAC** in the neighbour list:
the gateway is a neighbour on your link

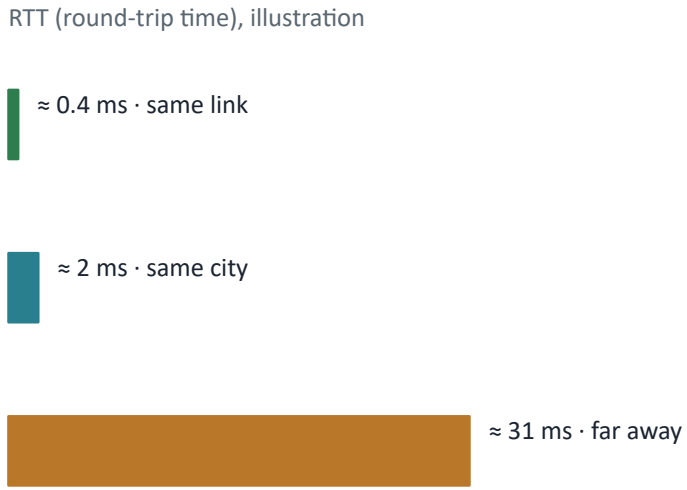
Q5 · which DNS servers? are they inside your own network?

Key idea Windows writes the default route as 0.0.0.0 with mask 0.0.0.0 — the same rule as Linux's "default via".

DNS (Domain Name System) · MAC (Media Access Control)

Task 3 · Reachability and round-trip time on Windows





Q9 variant: ping a colleague's Windows PC — often **no reply**

Q6 min / avg / max RTT and loss for each: most of the difference is **propagation**

Q7 min RTT 31 ms → one way ≤ 15.5 ms × 200 km/ms ≈ **3100 km**: an upper bound (path not straight, other delays); big sites serve from many places

Q8 `-l 1400` (larger packets): RTT rises only slightly — **transmission** delay depends on size

Q9 TTL (time to live) starts at 64, 128 or 255 and drops by one per router: reply ttl 52 → 64 − 52 = **12 routers**

Key idea Windows Firewall does not answer pings from other machines by default: a failed ping does not mean the machine is off.

PC (personal computer)

Task 4 · The path: tracert, Test-NetConnection, pathping

Windows tracert
ICMP (Internet Control Message Protocol) echo, like ping

Linux traceroute
UDP (User Datagram Protocol) packets to high ports



pathping: two phases (start it, then read on — it takes a minute or two)

1 · trace
list the routers, like tracert

→

2 · measure
send many probes to every hop: loss % per hop

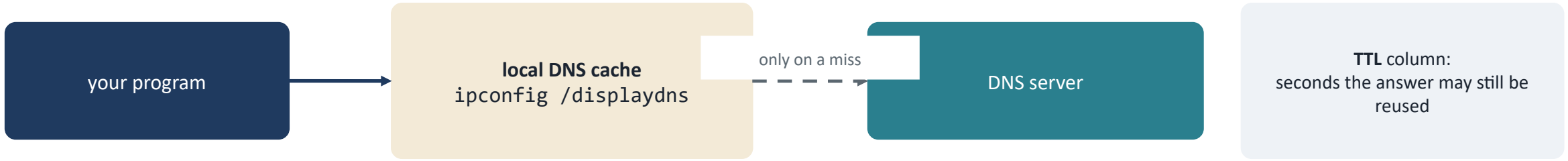
Q10 · TTL (time to live) start values: Linux and network equipment 64 or 255, **Windows 128**

Q13 · loss at a middle hop but **not** at the destination:
not a real problem — that router only rate-limits its replies; real loss would carry on to every later hop

Key idea Same TTL trick on both systems; different probe types can make the same path look different.

ISP (Internet service provider)

Task 5 · Names, addresses and the DNS cache



TTL column:
seconds the answer may still be reused

`Resolve-DnsName www.kernel.org`
A records (IPv4)

`… -Type AAAA`
IPv6 address record

`Resolve-DnsName <router address>`
reverse lookup

`Clear-DnsClientCache`
empty the cache

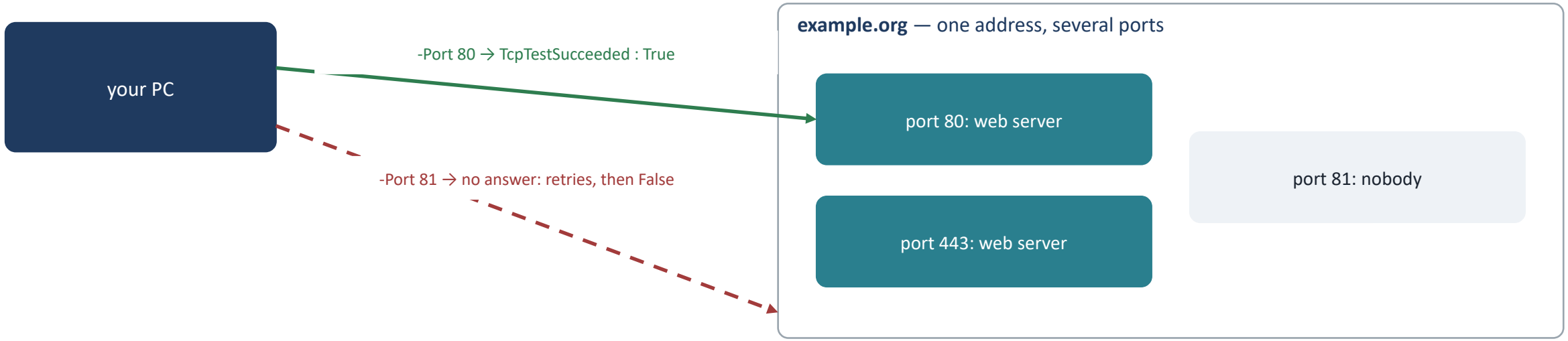
1 · find the name in the cache → 2 · Clear-DnsClientCache → 3 · check that it has gone

Q14 · addresses, IPv6 too? how long may the answer be reused?

Key idea The cache answers repeated questions locally until each record's TTL runs out.

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · DNS (Domain Name System) · TTL (time to live)

Task 6 · Ports: which program at that address?



Q15 · why does the port-81 test take so long?
nothing answers, so the connection request is **resent several times with growing waits** before giving up — about 20 s on Windows

```
curl.exe -v http://example.org/ -o NUL
```
> lines sent, < lines received — the HTTP exchange of the lecture

Key idea Address + port = one program on one host; a port with no listener costs a long timeout to discover.

HTTP (HyperText Transfer Protocol) · PC (personal computer)

Task 6 · A protocol by hand, and who owns each connection

```
$client = New-Object System.Net.Sockets.TcpClient('example.org', 80)
$stream = $client.GetStream()
$writer = New-Object System.IO.StreamWriter($stream)
$writer.NewLine = "`r`n"; $writer.AutoFlush = $true
$reader = New-Object System.IO.StreamReader($stream)
$writer.WriteLine('GET / HTTP/1.1')
$writer.WriteLine('Host: example.org')
$writer.WriteLine('Connection: close')
$writer.WriteLine('')
$reader.ReadToEnd(); $client.Close()
```

- open a TCP connection
- CR LF line endings
- send the request lines
- empty line ends it; read the reply

replace the first line by **HELLO**:
the reply becomes **400 Bad Request**

`netstat -ano`
connections + process numbers

→ `Get-NetTCPConnection -State Established`
LocalPort · RemoteAddress · OwningProcess

→ `Get-Process -Id <OwningProcess>`
→ the program, e.g. a browser

Key idea A connection is a pair of endpoints owned by processes — PowerShell maps them with OwningProcess (the process identifier).

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · CR (carriage return) · LF (line feed)

Task 7 · Throughput, watched live

```
curl.exe -o NUL -w "speed: %{speed_download} B/s   time: %{time_total} s\n" <test file URL>
```

Task Manager → Performance → adapter



compare
measured rate (Mbit/s) vs the adapter's **link speed** from Q1
alone, then with colleagues downloading together

Q16 · rate far below link speed and **falling when others download** → the shared lab uplink is the bottleneck;
steady and low whatever others do → probably the server

Key idea Measure throughput in Mbit/s and compare it with every candidate bottleneck on the path.

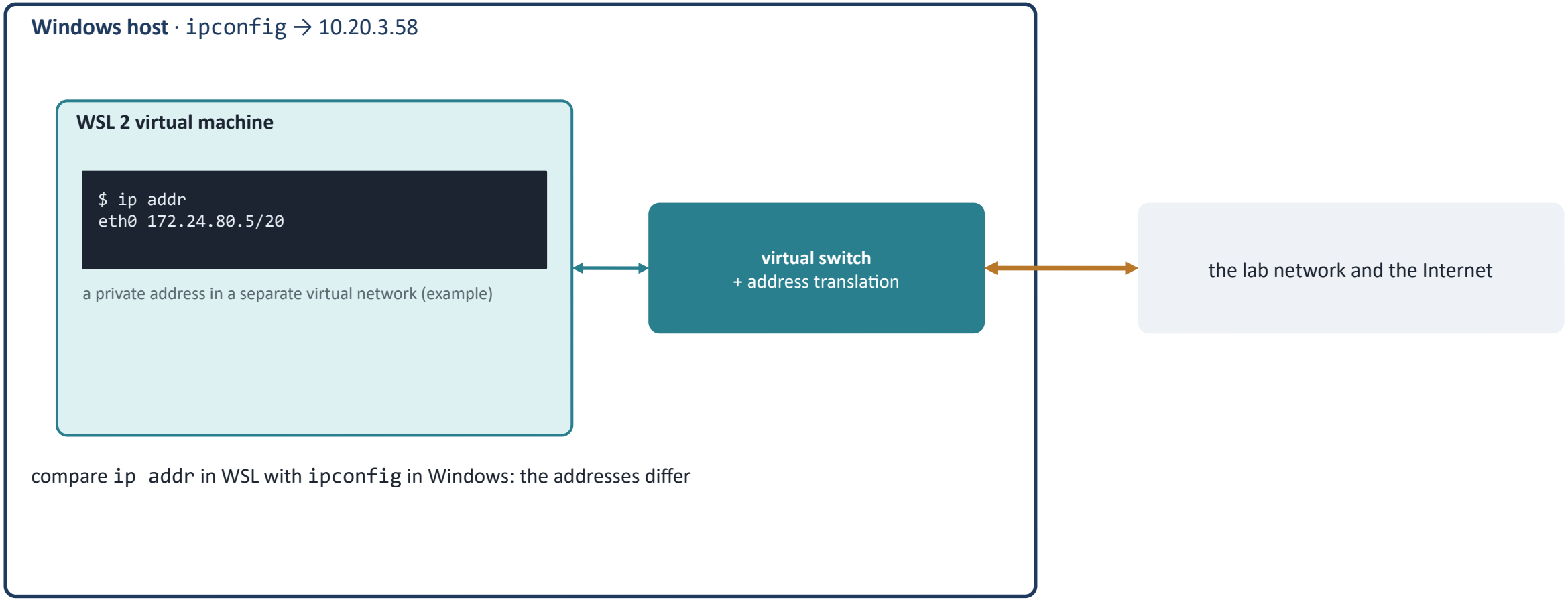
Task 8 · Linux and Windows side by side

purpose	Linux	Windows (classic)	Windows (PowerShell)
interfaces, addresses	`ip addr`	`ipconfig /all`	`Get-NetIPConfiguration`
forwarding table	`ip route`	`route print`	`Get-NetRoute`
route to one destination	`ip route get`	–	`Find-NetRoute`
neighbours on the link	`ip neigh`	`arp -a`	`Get-NetNeighbor`
reachability, RTT	`ping -c`	`ping -n`	`Test-Connection`
path	`traceroute, mtr`	`tracert, pathping`	`Test-NetConnection -TraceRoute`
name lookup	`dig`	`nslookup`	`Resolve-DnsName`
test a TCP port	`nc -vz host port`	–	`Test-NetConnection -Port`
open connections	`ss -tn`	`netstat -ano`	`Get-NetTCPConnection`
HTTP client	`curl -v`	`curl.exe -v`	`Invoke-WebRequest`
packet capture	`tcpdump, Wireshark`	`pktmon (admin), Wireshark`	–

Key idea Twelve purposes, three vocabularies: learn the purpose once, then look up the spelling.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · RTT (round-trip time) · mtr (my traceroute) · dig (domain information groper) · ss (socket statistics) · nc (netcat)

WSL 2: a Linux inside Windows, in its own network



Key idea Same machine, two hosts as far as the network is concerned: Windows on the lab network, WSL 2 behind it.

WSL (Windows Subsystem for Linux)

If something does not work (Windows)

`Request timed out` to every destination	no connectivity, or probes filtered: check `ipconfig` for an address and a gateway; ping the gateway first
address begins with 169.254	no address received automatically: check the cable or Wi-Fi; `ipconfig /renew`
ping to an address works, to a name does not	name resolution fails: check Q5; `Resolve-DnsName example.org -Server 1.1.1.1`
`curl -v` prints an HTML object, no trace	you ran the PowerShell alias — type `curl.exe`
“Access is denied” / “requires elevation”	open the terminal with “Run as administrator”

Key idea A 169.254 address means the host never got one: fix the link before anything else.

HTML (HyperText Markup Language)

Next week

Lecture 2

layered architectures: OSI (Open Systems Interconnection) and TCP/IP models, encapsulation step by step, delay, loss and throughput in depth

Seminar 2

queueing delay and the reference models: further problems

Laboratory 2

Wireshark: the headers of Figure 3 in real packets, on Linux and Windows

Key idea This week you met the network from outside; next week you open its packets.

TCP (Transmission Control Protocol) · IP (Internet Protocol)